Simulation and Visualisation of Vehicular Network Routing Protocols

Design, Implementation and Evaluation of a Linux-native OBU/RSU Simulator in Rust

---

Gonçalo Gomes

April 2026

0. Abstract

Vehicular networks present unique challenges for routing protocol design: nodes are highly mobile, connectivity is intermittent, and security requirements are stringent. This thesis presents the design, implementation, and evaluation of **vigilant-parakeet**, a Rust-based simulator and visualiser for experimenting with Layer-3 routing protocols in vehicular networks composed of On-Board Units (OBUs) and Road-Side Units (RSUs).

The simulator leverages Linux network namespaces to provide realistic, isolated per-node network stacks without requiring physical hardware. A heartbeat-driven routing protocol based on latency and hop-count metrics is implemented, with support for N-best candidate caching and rapid upstream failover. A full security architecture provides end-to-end encrypted OBU–server communication via a configurable key exchange (classical X25519 or quantum-resistant ML-KEM-768), HKDF-derived session keys, and AEAD payload encryption (AES-256-GCM, AES-128-GCM, or ChaCha20-Poly1305); an optional digital signature layer (Ed25519 or ML-DSA-65) defends the key exchange against man-in-the-middle attacks in both Trust-on-First-Use and pre-registered PKI deployment modes. A signed session-revocation mechanism allows the server to forcibly terminate and re-key any OBU session, protected against replay by a timestamp-and-nonce scheme. The system exposes an HTTP control API and a browser-based visualisation dashboard for interactive experimentation.

Key contributions include: a modular Rust workspace architecture separating shared protocol logic from node-specific implementations; a configurable cipher suite with transparent relay opacity ensuring intermediate RSUs cannot access session keys; post-quantum security via ML-KEM-768 key encapsulation and ML-DSA-65 digital signatures (NIST FIPS 203/204), addressing harvest-now- decrypt-later threats; a HeartbeatReply replay-detection window following the IPsec AH design; an in-process programmable hub for deterministic, reproducible testing without root privileges; and an evaluation of routing behaviour under varying latency and packet-loss conditions.

0. Acknowledgements

I would like to thank everyone who supported this work.

0. Table of Contents

1 Introduction . . . 1
1.1 Motivation . . . 1
1.2 Research Questions . . . 1
1.3 Contributions . . . 1
1.4 Thesis Structure . . . 2
2 Background . . . 4
2.1 Vehicular Networks . . . 4
2.1.1 Architecture: OBU and RSU . . . 4
2.1.2 Communication Modes . . . 4
2.1.3 Channel Characteristics . . . 4
2.1.4 Physical Layer Technologies . . . 5
2.2 V2X Application Domains and QoS Requirements . . . 5
2.2.1 Safety-Critical Applications . . . 5
2.2.2 Traffic Efficiency Applications . . . 6
2.2.3 Infotainment and Value-Added Services . . . 6
2.2.4 Cooperative Perception for Automated Driving . . . 7
2.2.5 QoS Implications for Routing and Security . . . 7
2.3 Routing in Vehicular Networks . . . 7
2.3.1 Topology-Based Proactive Routing . . . 7
2.3.2 Topology-Based Reactive Routing . . . 8
2.3.3 Position-Based Routing . . . 9
2.3.4 Delay-Tolerant Routing . . . 10
2.3.5 Cluster-Based Routing . . . 10
2.3.6 Beacon / Heartbeat Routing . . . 10
2.3.7 Comparative Analysis . . . 11
2.4 Security in Vehicular Networks . . . 12
2.4.1 Attack Taxonomy . . . 12
2.4.2 PKI-Based Security: IEEE 1609.2 and ETSI ITS . . . 13
2.4.3 Pseudonym Management . . . 14
2.4.4 Broadcast Authentication . . . 14
2.4.5 Misbehavior Detection and Revocation . . . 15
2.4.6 Cryptography Primer . . . 16
2.4.7 Post-Quantum Cryptography . . . 17
2.4.8 Data Confidentiality . . . 19
2.4.9 Replay Window Mechanism . . . 20
2.4.10 The Reference Security Model . . . 21
2.5 Simulation Approaches . . . 21
2.5.1 Network Simulators . . . 21

2.5.2 Container and Namespace-Based Simulation . . . 22
2.5.3 Rust for Systems Programming . . . 23
3 Related Work . . . 26
3.1 VANET Simulation and Emulation Platforms . . . 26
3.1.1 Coupled Mobility and Network Simulators . . . 26
3.1.2 Emulation-Based Approaches . . . 27
3.2 Routing Protocol Implementations . . . 29
3.3 VANET Security Frameworks . . . 30
3.3.1 Deployed Credential Management Systems . . . 30
3.3.2 Research Prototypes and Security Analysis . . . 30
3.3.3 Post-Quantum Security in Vehicular Networks . . . 31
3.4 Summary and Positioning . . . 31
4 Architecture . . . 33
4.1 Workspace Overview . . . 33
4.2 Three-Tier Network Model . . . 33
4.3 Crate Responsibilities . . . 34
4.3.1 `common` — OS Abstractions . . . 34
4.3.2 `node_lib` — Protocol Building Blocks . . . 34
4.3.3 Internal Async Task Architecture . . . 35
4.3.4 `obu_lib` — OBU Node . . . 36
4.3.5 `rsu_lib` — RSU Node . . . 37
4.3.6 `server_lib` — Server Node . . . 37
4.3.7 `node` — Single-Node Binary . . . 37
4.3.8 `simulator` — Multi-Node Orchestration . . . 37
4.3.9 `visualization` — Browser Dashboard . . . 38
4.3.10 `scripts_tools` — Experiment Analysis CLI . . . 38
4.3.11 `keygen` — Key-Generation Utility . . . 38
4.4 Simulator Architecture in Depth . . . 39
4.4.1 Network Namespace Isolation . . . 39
4.4.2 Channel Model . . . 39
4.4.3 Node Lifecycle . . . 40
4.4.4 TUI Dashboard . . . 40
4.4.5 HTTP Control API . . . 41
4.5 Key Design Decisions . . . 42
4.5.1 RSU Opacity . . . 42
4.5.2 Separation of Protocol Logic from Orchestration . . . 42
4.5.3 Pure Route Selection . . . 42
4.5.4 Async-First, No Threads per Node . . . 42
4.5.5 Feature-Gated Instrumentation . . . 43
4.5.6 Test Infrastructure Without Root . . . 43
4.5.7 Why Userspace Channel Emulation over `tc-netem` . . . 43
5 Implementation . . . 45
5.1 Wire Protocol . . . 45
5.1.1 Message Types . . . 45
5.1.2 Heartbeat Wire Layout . . . 45
5.1.3 Cloud Protocol Wire Layout . . . 46

5.2 Routing Protocol . . . 46
5.2.1 Heartbeat Emission (RSU) . . . 46
5.2.2 Heartbeat Reception and Route Building (OBU) . . . 46
5.2.3 Route Selection Metric . . . 46
5.2.4 N-Best Candidate Caching . . . 47
5.2.5 Loop Prevention . . . 48
5.2.6 Replay Detection . . . 48
5.3 End-to-End Data Path . . . 49
5.4 Simulator Orchestration . . . 49
5.4.1 Network Namespace Setup . . . 49
5.4.2 `node_factory` Module . . . 50
5.4.3 HTTP Control API (feature: `webview`) . . . 50
5.5 TCP Admin Console . . . 50
5.5.1 OBU Admin Commands . . . 51
5.5.2 RSU Admin Commands . . . 51
5.5.3 Server Admin Commands . . . 51
5.6 Visualisation Dashboard . . . 52
5.6.1 Technology Stack . . . 52
5.6.2 Dashboard Components . . . 52
5.6.3 Architectural Separation . . . 53
5.7 Simulator Configuration . . . 53
5.7.1 Topology File Format . . . 53
5.7.2 Node Configuration File Format . . . 54
5.8 Test Infrastructure . . . 55
5.8.1 The `Hub` Helper . . . 55
5.8.2 TUN Shim . . . 56
5.8.3 Integration Test Coverage . . . 56
6 Security Architecture . . . 58
6.1 Threat Model . . . 58
6.2 Configurable Cipher Suite . . . 59
6.3 Key Exchange Protocol . . . 59
6.3.1 Wire Format . . . 59
6.3.2 Handshake Flow . . . 60
6.4 Key Derivation . . . 62
6.5 Symmetric Encryption . . . 62
6.6 DH Key Store Lifecycle . . . 63
6.7 Authentication for Key Exchange . . . 64
6.7.1 Signing Identity . . . 65
6.7.2 Signature Computation and Verification . . . 65
6.7.3 Authenticated Handshake Flow . . . 66
6.7.4 Trust Models . . . 66
6.7.5 Security Properties Summary . . . 67
6.8 Session Revocation . . . 68
6.8.1 Wire Format . . . 68
6.8.2 Replay Prevention . . . 68
6.9 RSU Opacity . . . 69

6.10 Comparison to Related Security Approaches ... 69
6.10.1 Comparison with IEEE 1609.2 / PKI-Based Approaches ... 69
6.10.2 Comparison with TESLA-Based Broadcast Authentication ... 70
6.10.3 Comparison with End-to-End Encryption Schemes ... 70
6.10.4 Summary ... 70
6.11 Attack Surface Coverage ... 71
7 Evaluation ... 74
7.1 Experimental Setup ... 74
7.1.1 Hardware and Software Configuration ... 74
7.1.2 Measurement Methodology ... 74
7.1.3 Topology Configurations ... 75
7.1.4 Baseline and Variant Comparisons ... 75
7.2 Route Convergence ... 75
7.3 Route Quality Under Packet Loss ... 76
7.4 N-Best Failover Latency ... 77
7.5 Memory and CPU Overhead ... 77
7.5.1 Memory Overhead of N-Best Caching ... 78
7.5.2 CPU Utilisation ... 78
7.6 Cryptographic Session Establishment ... 78
7.6.1 Key Exchange Latency ... 78
7.6.2 Session Revocation Overhead ... 79
7.7 Discussion ... 79
7.7.1 Routing Behaviour ... 79
7.7.2 Failover Behaviour ... 80
7.7.3 Cryptographic Overhead ... 80
7.7.4 Limitations ... 80
8 Conclusion ... 81
8.1 Summary ... 81
8.2 Reflection on Design Decisions ... 82
8.3 Limitations ... 82
8.4 Future Work ... 83
References ... 85

0. List of Figures

Figure 1 Cargo workspace dependency graph 33
Figure 2 Three-tier network architecture 34
Figure 3 End-to-end data path across all three tiers 49
Figure 4 Key exchange handshake (unsigned): X25519 ECDH or ML-KEM-768 KEM, and subsequent encrypted data flow 62
Figure 5 DH key store state machine per peer 63
Figure 6 Authenticated handshake with Ed25519 or ML-DSA-65 signatures and optional PKI enforcement 66

0. List of Listings

1. Introduction

1.1 Motivation

Vehicular networks — also known as VANETs (Vehicular Ad-hoc Networks) — are a class of mobile ad-hoc network in which vehicles and roadside infrastructure communicate to improve road safety, traffic efficiency, and passenger experience. The combination of high node mobility, intermittent connectivity, and safety-critical applications makes routing a particularly demanding problem in this domain.

Existing simulation tools either require costly proprietary hardware, operate at the application layer only, or fail to model the per-link latency and packet loss that characterise real vehicular channels. This thesis addresses that gap by presenting **vigilant-parakeet**: a Linux-native, open-source simulator that runs real Layer-3 node logic inside isolated network namespaces, allowing researchers to study routing protocols under controlled but realistic network conditions.

The project was originally motivated by the vehicular-network security work of [1], whose routing model forms the basis of the protocol implemented here.

1.2 Research Questions

This thesis investigates the following questions:

1. Can a single-machine Linux simulator faithfully reproduce the routing dynamics of a multi-hop vehicular network without specialised hardware?
2. How does the choice of routing metric (latency vs. hop count) affect convergence time and route stability under varying loss conditions?
3. What is the overhead introduced by N-best candidate caching on memory and CPU, and how much does it reduce route-restoration latency after a next-hop failure?
4. What is the practical overhead of post-quantum key exchange (ML-KEM-768
   1. ML-DSA-65) in terms of handshake latency and message size, and is it

   compatible with the latency budgets of vehicular V2I sessions?

1.3 Contributions

The primary contributions of this work are:

- **A modular Rust simulator** (`vigilant-parakeet`) built as a Cargo workspace, separating shared protocol logic (`node_lib`) from node-type implementations (`obu_lib`, `rsu_lib`) and the orchestration layer (`simulator`).
- **A heartbeat-based routing protocol** with latency-aware metric computation, N-best upstream candidate caching, and fast failover.
- **An end-to-end security architecture** providing encrypted OBU–server communication via a configurable key exchange (classical X25519 or quantum-resistant ML-KEM-768, NIST FIPS 203), HKDF-derived session keys, and AEAD payload encryption (AES-256-GCM, AES-128-GCM, ChaCha20-Poly1305). An optional digital signature layer (Ed25519 or ML-DSA-65, NIST FIPS 204) protects the key exchange itself, supporting both Trust-on-First-Use (TOFU) and pre-registered PKI deployment modes. The post-quantum combination of ML-KEM-768 and ML-DSA-65 addresses harvest-now-decrypt-later threats against long-lived vehicular infrastructure.
- **A HeartbeatReply replay-detection mechanism** (`rsu_lib::ReplayWindow`) at RSUs — a per-sender sliding 64-bit bitmask window following the IPsec AH design — preventing stale routing state from being injected via replayed control messages. A supplementary window-poisoning defence prevents forged large sequence numbers from rendering the window permanently closed.
- **A signed session-revocation protocol** allowing the server to forcibly terminate an OBU's established DH session and trigger immediate re-keying. Revocation messages carry a timestamp and a fresh random nonce, with the OBU maintaining a time-bounded nonce cache to prevent replay over the revocation validity window.
- **A `keygen` utility** for generating Ed25519 and ML-DSA-65 signing keypairs at provisioning time, enabling stable, restartable node identities for PKI mode deployments.
- **An in-process test hub** (`node_lib::test_helpers::hub::Hub`) enabling deterministic, reproducible integration testing without root privileges or physical network devices.
- **A browser-based visualisation dashboard** consuming a real-time HTTP metrics API exposed by the simulator.
- **An empirical evaluation** of routing behaviour across a range of topologies, latency profiles, and packet-loss rates.

1.4 Thesis Structure

The remainder of this thesis is organised as follows.

- Section 2 reviews vehicular networking concepts, relevant routing protocols, post-quantum cryptographic foundations, and simulation approaches.
- Section 3 surveys related VANET simulation platforms, routing protocol implementations, and security frameworks, positioning vigilant-parakeet within the existing literature.

- Section 4 describes the high-level design of vigilant-parakeet and the rationale behind its crate decomposition.
- Section 5 details the implementation of the routing protocol, the simulator, the test infrastructure, and the visualisation layer.
- Section 6 presents the security architecture: threat model, DH key exchange (X25519 and ML-KEM-768), HKDF key derivation, configurable AEAD cipher suite, DH key store lifecycle, the Ed25519/ML-DSA-65 authentication layer with its TOFU and PKI trust models, HeartbeatReply replay detection, and the signed session-revocation protocol.
- Section 7 presents the experimental setup and results.
- Section 8 summarises the findings, reflects on limitations, and proposes directions for future work.

2. Background

2.1 Vehicular Networks

2.1.1 Architecture: OBU and RSU

A vehicular network consists of two classes of node:

- **On-Board Units (OBUs)** are carried by vehicles. They are mobile, battery-powered, and connect to the network only while within radio range of other nodes.
- **Road-Side Units (RSUs)** are fixed infrastructure nodes deployed at intersections or along roadsides. They act as gateways to the wider internet and as anchors for the mobile OBUs.

This architecture is standardised under the ETSI ITS (Intelligent Transport Systems) framework [2] and the IEEE WAVE (Wireless Access in Vehicular Environments) suite [3].

2.1.2 Communication Modes

Two communication modes are defined:

- **V2I (Vehicle-to-Infrastructure)**: direct communication between an OBU and a nearby RSU.
- **V2V (Vehicle-to-Vehicle)**: direct communication between two OBUs, typically used when no RSU is in range or for cooperative awareness.

Multi-hop forwarding allows OBUs beyond direct RSU range to reach the network through intermediate OBU relays, forming an ad-hoc mesh.

2.1.3 Channel Characteristics

Vehicular channels exhibit:

- **High Doppler spread** due to relative velocities up to 200 km/h.
- **Rapid topology change** as vehicles enter and leave radio range within seconds.
- **Asymmetric links** where forward and reverse path quality can differ significantly.
- **Intermittent connectivity** making connection-oriented protocols impractical for many use cases.

These properties motivate the use of lightweight, beacon-based routing rather than heavy link-state or distance-vector protocols.

2.1.4 Physical Layer Technologies

Two dominant air-interface standards have been deployed for vehicular communications:

DSRC (Dedicated Short-Range Communications) [4] is a licensed 5.9 GHz band (75 MHz in the US; 30 MHz in Europe under ITS-G5) based on IEEE 802.11p (WAVE). It provides peer-to-peer broadcast with low latency (typically below 10 ms) and ranges up to approximately 1 km, making it well-suited for safety applications. Its infrastructure-less operation allows OBUs to communicate without pre-existing cellular coverage. The standard has been in commercial deployment in the US, Europe, and Japan since the early 2010s.

C-V2X (Cellular Vehicle-to-Everything) [5] encompasses both LTE-V2X (3GPP Release 14/15) and NR-V2X (Release 16, 5G). C-V2X Mode 4 operates in a sidelink configuration that does not require network infrastructure coverage, competing directly with DSRC at the physical layer. 5G NR-V2X adds unicast and groupcast transmission modes enabling latency-sensitive interactions beyond what broadcast-only DSRC supports. The two standards are not interoperable at the physical layer, and their relative merits — reliability under congestion, non-line-of-sight range, infrastructure cost, standardisation maturity — remain an active technical and regulatory debate.

The protocols implemented in vigilant-parakeet operate at Layer 3 and above and are agnostic to the specific air interface: any link delivering packets between neighbouring nodes with the appropriate latency and loss characteristics is a valid substrate.

2.2 V2X Application Domains and QoS Requirements

The design of a VANET routing and security layer cannot be evaluated in isolation from the applications it carries. Each V2X application class places qualitatively different demands on the network in terms of latency, reliability, and message rate, which in turn determine the feasibility of various routing and security mechanisms. [6] surveys the major application categories.

2.2.1 Safety-Critical Applications

Safety applications are the primary driver of the DSRC/IEEE 802.11p standard and represent the most demanding QoS class. They include:

Cooperative Collision Avoidance (CCA) OBUs broadcast position, speed, and heading at 10 Hz. Receiving vehicles detect trajectories that will intersect within a collision horizon and warn the driver or apply automatic braking. ETSI ITS [7] specifies an end-to-end latency budget of 100 ms for safety-critical alerts, with the radio access layer alone capped at 10–50 ms.

Intersection Management RSUs broadcast signal phase and timing (SPAT) and map data (MAP) messages to approaching vehicles. Latency of tens of milliseconds allows OBUs to optimise speed to catch a green phase (green wave) or to brake before a red one.

Emergency Vehicle Notification police, ambulance, and fire vehicles broadcast priority alerts. Relay OBUs forward the alert on DSRC; infrastructure RSUs relay to out-of-range areas. Latency requirements are below 500 ms over a 300 m radius.

Post-Crash Notification a crashed vehicle broadcasts an eCall trigger to nearby OBUs, which relay it to an RSU for forwarding to emergency services. The V2X path serves as a backup when the in-vehicle cellular modem is inoperative or blocked by metalwork.

All safety applications share the properties of: small message payloads (CAM messages are typically 200–400 bytes); broadcast or geocast delivery (authenticated but not encrypted under IEEE 1609.2, since the payload is meant to be received by any nearby vehicle); and strict latency requirements (typically 10–100 ms) that rule out reactive route discovery protocols.

2.2.2 Traffic Efficiency Applications

Traffic efficiency applications operate on a seconds-to-minutes time scale and can tolerate somewhat higher latency than safety applications in exchange for coordinated vehicle behaviour:

Cooperative Adaptive Cruise Control (CACC) vehicles in a platoon exchange acceleration and braking intentions at 10–50 Hz. CACC allows inter-vehicle gaps as short as 0.3–1 s at highway speeds by providing advance notice of upstream deceleration, allowing following vehicles to brake before the deceleration propagates backward through the platoon. Latency requirements are 10–20 ms for effective gap closing [8].

Truck Platooning multiple trucks drive in close convoy under joint electronic control. The lead truck broadcasts velocity commands; followers adjust throttle and brakes via V2V. Since the platoon can span several hundred metres, multi-hop relaying with controlled latency is required. End-to-end latency must be bounded: unbounded latency variance causes platoon instability and potential collision. This is precisely the use case for the N-best upstream caching and latency-based routing metric in vigilant-parakeet.

Traffic Signal Phase Optimisation RSUs can receive aggregate traffic density information from approaching OBUs and adapt signal timing accordingly. Unicast or groupcast delivery via V2I with latency on the order of one signal phase (tens of seconds) is sufficient.

2.2.3 Infotainment and Value-Added Services

Non-safety applications carry internet traffic, parking availability data, map updates, and digital content downloads from infrastructure RSUs. These are typically unicast TCP sessions requiring reliable delivery but tolerating latency on the order of hundreds of milliseconds to seconds. The OBU acts as a mobile gateway, and the key routing challenge is hand-off: maintaining session continuity as the vehicle moves between RSU coverage areas. The DH session key architecture in vigilant-parakeet is designed for this use case:

the session key is between the OBU and a backend server, not the OBU and the current RSU, so RSU hand-off does not require rekeying.

2.2.4 Cooperative Perception for Automated Driving

Emerging autonomous driving applications share raw sensor data — camera images, LiDAR point clouds, radar returns — between nearby vehicles and roadside sensors to extend beyond each vehicle's direct sensor horizon [9]. This data sharing (Collective Perception Messages, CPMs) demands very high throughput (potentially tens of Mbps per vehicle) and low latency (50–100 ms for useful fusion). These requirements push beyond what 802.11p can deliver at vehicular densities, motivating 5G NR-V2X sidelink for this application class. The routing and security protocols studied in vigilant-parakeet are relevant to the control plane of such systems (session establishment, relay selection) even if the data plane would require different transport.

2.2.5 QoS Implications for Routing and Security

Application class	Latency budget	Reliability	Payload	Encrypted?
Safety (CAM, DENM)	<100 ms	99.999%	200–400 B	No (signed)
CACC / platooning	<20 ms	99.99%	100–200 B	Optional
Traffic efficiency	<1 s	99%	0.5–2 KB	Optional
Infotainment / data	<5 s	95%	large	Yes
Cooperative perception	50–100 ms	99.9%	MB-range	Yes

Table 1 — QoS requirements across V2X application classes

The table illustrates why the routing metric matters: for CACC and platooning, route oscillation of even a few tens of milliseconds per interval is disruptive. The 10% hysteresis threshold in vigilant-parakeet's route selection (see Section 5.2) is calibrated to prevent exactly this oscillation. For safety applications, the absence of payload encryption is a deliberate design choice in the standards: CAM messages are public safety information, and encryption would prevent roadside sensors and other non-paired receivers from using the broadcast. The data-plane encryption in vigilant-parakeet applies to the infotainment-class unicast sessions that require confidentiality.

2.3 Routing in Vehicular Networks

A survey of routing approaches in VANETs [10] identifies three broad families: topology-based (proactive and reactive), position-based, and infrastructure-assisted beacon routing. Each presents different trade-offs under vehicular channel conditions. More recently, delay-tolerant and cluster-based approaches have been studied for sparse or heterogeneous networks.

2.3.1 Topology-Based Proactive Routing

Proactive protocols maintain routing tables continuously via periodic control exchanges, so that a route is available immediately when a packet must be sent.

DSDV (Destination-Sequenced Distance-Vector) [11] extends the classical Bellman-Ford algorithm with sequence-numbered advertisements to eliminate count-to-infinity loops. Each destination owns a monotonically increasing sequence number; a node receiving two routes for the same destination keeps the one with the higher sequence number (more recent), breaking ties by lower cost. DSDV transmits both periodic full routing-table dumps and event-triggered incremental updates when a link cost changes. In vehicular environments the rate of topology change frequently forces full dumps, generating control overhead that scales as $O(N^2)$ in the number of nodes. Convergence after a link break requires a new advertisement to propagate across the network, during which stale routes persist.

OLSR (Optimised Link State Routing) [12], an IETF standard for MANETs, reduces link-state flooding overhead through **multipoint relay** (MPR) selection. Each node selects a minimal subset of its one-hop neighbours that collectively cover all its two-hop neighbours. Only MPRs retransmit topology control (TC) messages, pruning the broadcast spanning tree from $O(N^2)$ to significantly fewer copies. Each node's MPR selector set is reported in TC messages, allowing every node to reconstruct the global topology and run Dijkstra for shortest-path computation. OLSR is well-studied for MANETs but problematic for high-speed VANETs: the MPR set computed for one epoch may be entirely invalid a second later as vehicles move apart, causing OLSR to route through non-existent links and generate frequent topology updates that can exceed the capacity of the control channel.

2.3.2 Topology-Based Reactive Routing

Reactive protocols generate routes on demand, reducing steady-state control overhead at the cost of discovery latency.

AODV (Ad-hoc On-Demand Distance Vector) [13] discovers routes by flooding a Route Request (RREQ) packet. RREQ propagation is controlled by an expanding ring search: the TTL field in the IP header starts at a small value and is doubled on each successive retry if no reply is received, limiting initial broadcast scope. The destination — or any intermediate node with a fresher cached route — responds with a Route Reply (RREP) along the reverse path recorded during RREQ propagation. Active routes are maintained by periodic Hello beacons between next-hop neighbours; when a link break is detected (Hello timeout or transmission failure), Route Error (RERR) messages propagate upstream to invalidate affected routing table entries. AODV maintains per-destination soft state rather than a global topology map, reducing memory requirements.

DSR (Dynamic Source Routing) [14] eliminates per-hop routing tables entirely by embedding the complete source-to-destination path in the packet header. Route discovery is identical in structure to AODV: a Route Request floods the network, and intermediate nodes append their address to a route-record field. The source caches the discovered path and uses it for all subsequent packets to the same destination. DSR benefits from **promiscuous caching**: a node that overhears a Route Reply passing by caches the

discovered path even though it was not the requester, building up a rich route cache at low control cost. The cost is larger packet headers proportional to path length and cache staleness: cached routes are not proactively invalidated and may persist well past the point where the path is still valid.

In vehicular settings, both AODV and DSR suffer from the route discovery round-trip exceeding the RSU contact window, making reactive protocols unsuitable for delay-sensitive applications. AODV's RREQ flooding also produces the **broadcast storm** problem in dense networks: simultaneous retransmission by many neighbours causes massive collision at the receiver.

2.3.3 Position-Based Routing

Geographic routing exploits GPS coordinates to forward packets toward a destination without maintaining global topology state. GPSR (Greedy Perimeter Stateless Routing) [15] is the seminal work.

Greedy forwarding: each hop selects the neighbour geographically closest to the destination — i.e., whose Euclidean distance to the destination is smaller than the forwarding node's own distance. Only one-hop neighbour positions need be maintained, via periodic position beacons. A packet makes progress toward the destination at each hop without the sender knowing the full path.

Perimeter mode: when no closer neighbour exists (a routing void caused by a hole in coverage), GPSR switches to planar graph traversal. The network graph is planarised using either a Gabriel Graph (GG) or Relative Neighbourhood Graph (RNG) — both remove edges whose midpoints are covered by another node. The right-hand rule is then applied to traverse the void perimeter: packets always take the next edge clockwise relative to the arrival direction, guaranteed to eventually escape the void and resume greedy forwarding. GPSR alternates between greedy and perimeter modes as packets cross different regions.

Limitations: GPSR requires accurate, frequently-updated position information via position beacons. In vehicular environments, positions change fast enough that beacons from a second ago may represent an outdated topology. The planarisation step also assumes symmetric links; VANET channels are frequently asymmetric, which invalidates the planarity guarantee and can cause GPSR to loop in perimeter mode. Furthermore, straight-line geometric proximity is a poor predictor of radio reachability in urban environments where buildings obstruct paths.

Road-topology-aware variants: GPCR (Greedy Perimeter Coordinator Routing) [16] targets urban environments by using road intersections as preferred forwarders. Rather than pure Euclidean proximity, the routing metric favours nodes that lie on the road segment leading toward the destination. Perimeter traversal follows road graph edges rather than a planar graph extracted from unreliable position data. A-STAR [17] augments this with an attractiveness metric derived from bus-route maps: packets prefer roads with higher expected vehicle density, improving delivery ratio in sparse urban networks by biasing toward well-trafficked corridors where relay opportunities are frequent.

2.3.4 Delay-Tolerant Routing

When end-to-end connectivity cannot be guaranteed — sparse rural VANETs, infrastructure gaps, or highly intermittent channels — **store-carry-forward** (Delay-Tolerant Network, DTN) approaches relax the requirement for a path to exist at the time of transmission [18]. A vehicle carries a message in its buffer as it drives, and delivers when it comes within range of an appropriate relay or the final destination.

VADD (Vehicle-Assisted Data Delivery) [18] models each road intersection as a choice point. At each intersection, the forwarding vehicle selects the adjacent road segment with minimum **expected packet delivery delay**, computed from average vehicle speed, traffic density, and segment length. This is equivalent to shortest-path computation on a mobility-weighted road graph rather than an instantaneous connectivity graph. VADD's key insight is that road-level mobility statistics — accessible from traffic databases or historical traces — are more stable and predictable than instantaneous V2V topology. Its limitation is dependence on accurate speed models; sudden traffic events (accidents, road closures) can invalidate predictions and cause packets to be carried on suboptimal trajectories.

2.3.5 Cluster-Based Routing

In dense networks, flat flooding of control messages is inefficient. Cluster- based approaches [19] organise nodes into groups with a **cluster head** (CH) aggregating routing state and communicating with other CHs via an inter-cluster backbone. Member nodes route through their CH, reducing the number of nodes participating in global route computation.

The central challenge in vehicular clustering is stability: high relative mobility causes frequent CH elections and member migration. Stability metrics such as relative velocity between member and CH candidate, transmission range overlap, and connection duration are used to select CHS likely to remain in range [19]. Despite this, cluster lifetime in freeway scenarios is typically tens of seconds — far shorter than in static MANETs — limiting the benefit of cluster-level state aggregation. Cluster-based routing is most effective in relatively stable scenarios such as bus fleets or convoy formations with predictable mobility.

2.3.6 Beacon / Heartbeat Routing

Infrastructure-assisted vehicular networks use periodic **heartbeat** or **beacon** messages emitted by RSUs to drive route selection. OBUs overhearing heartbeats measure propagation delay, record hop count, and build upstream routing tables toward the emitting RSU. This approach suits V2I settings where connectivity to a fixed RSU is the primary goal: the routing problem reduces to selecting the best relay path toward a known, stationary gateway.

The metric for path selection varies across implementations. Hop-count alone is simple but insensitive to per-link quality differences; RTT-based metrics better reflect actual channel conditions but require reply messages and may exhibit oscillation under asymmetric loss.

Hybrid metrics — combining a latency component with a hop-count fallback — are common in practice and are the basis of the routing protocol implemented in this work (see Section 5.2).

2.3.7 Comparative Analysis

Protocol family	**State maintained**	**Control overhead**	**Route latency**	**VANET suitability**
Proactive (DSDV, OLSR)	Global topology	High — continuous $O(N^2)$ updates	Zero	Poor — overhead scales with density; high churn
Reactive (AODV, DSR)	Per-route soft state	Low (on demand)	High — discovery flood may outlast RSU contact window	Poor — broadcast storm; discovery latency incompatible with vehicular contacts
Position-based (GPSR, GPCR)	Neighbour positions only	Low — position beacons	Zero	Moderate — void problem; urban adaptation needed; GPS accuracy required
DTN (VADD)	Local buffer + road map	Minimal	Unbounded — carry-forward	Good for sparse/intermittent networks; unsuitable for low-latency applications
Cluster-based	Cluster topology	Moderate — CH elections	Low within cluster	Limited — short cluster lifetime in fast vehicular networks
Beacon/heartbeat **(this work)**	Per-RSU upstream tables	Low — RSU-driven periodic beacons	Zero for established routes	Good for V2I: RSU reachability is proactive; metric sensitive to link quality

Table 2 — Summary comparison of VANET routing families

2.4 Security in Vehicular Networks

2.4.1 Attack Taxonomy

Vehicular networks are exposed to a range of adversarial behaviours spanning both the data plane (payload forwarding) and the control plane (routing). The following taxonomy covers the main attack classes studied in the literature and addressed, in whole or in part, by the security architecture implemented in this work.

- **Routing table poisoning**: an adversary controlling one or more intermediate nodes selectively drops, replays, or crafts heartbeat messages to corrupt routing state across a wide area, attracting traffic through attacker-controlled paths. [1] provides a detailed analysis of this class of attack in the OBU/RSU architecture. Defences require authenticated heartbeats (see Section 2.4.4) and sequence-number freshness checks.
- **Black hole and grey hole attacks**: a node advertises an artificially attractive route then silently drops (black hole) or probabilistically forwards (grey hole) the attracted traffic. Detection relies on neighbourhood monitoring: a watchdog node in promiscuous mode can confirm whether its next hop forwarded the packet it received [20]. The CONFIDANT protocol [21] extends watchdog monitoring with a distributed reputation system in which nodes share misbehaviour observations and route around nodes with poor accumulated reputation.
- **Sybil attacks** [22]: a single physical node presents multiple false MAC or IP identities, polluting routing tables and consensus mechanisms with phantom participants. Sybil identities are particularly harmful in heartbeat-based routing because each fake identity can independently attract traffic and manipulate route scores. Certificate-based identity binding (IEEE 1609.2, Section 2.4.2) mitigates Sybil attacks under the assumption that a PKI cannot issue unbounded certificates to a single physical node.
- **Wormhole attacks** [23]: two colluding nodes tunnel packets between distant network segments out-of-band, creating the illusion of a low-latency shortcut. Neighbours at both tunnel endpoints observe apparently direct connectivity, preferring the wormhole path; the colluding pair can then selectively manipulate forwarded traffic. Detection approaches include packet leashes [23] — geographic leashes embed GPS position and timestamp; temporal leashes bound transmission delay using synchronised clocks — and directional-antenna methods, but both require additional hardware or tight clock synchronisation.
- **Replay attacks**: previously captured control messages (e.g., heartbeats with high sequence numbers) are re-injected to maintain stale routing entries or to exhaust sequence counters. Monotone sequence numbers combined with short validity windows (one heartbeat interval) bound replay exposure.
- **Passive eavesdropping**: a physically present adversary captures payload frames from any link, recovering application data from unencrypted transmissions. Addressed by end-to-end AEAD encryption with session keys never held by relay nodes (see Section 6).

- **Position spoofing**: a node broadcasts false GPS coordinates to manipulate position-based routing decisions. Cross-checking claimed positions using time-of-arrival measurements from multiple anchor RSUs can detect inconsistencies indicating fabricated positions; this remains an open problem for single-RSU scenarios.

2.4.2 PKI-Based Security: IEEE 1609.2 and ETSI ITS

IEEE 1609.2 [24] defines the security architecture for Wireless Access in Vehicular Environments (WAVE). The architecture is a tiered PKI consisting of:

- **Root CA**: the trust anchor. Its certificate is embedded in vehicle firmware at manufacture time. It signs the certificates of Intermediate CAs and is kept offline to limit exposure.
- **Intermediate CAs / Policy CAs**: issue certificates to lower-level authorities with scoped permissions expressed as **service-specific permissions** (SSP) bit strings. A certificate for a given ITS application (safety messaging, intersection management, etc.) carries only the SSP for that application.
- **Pseudonymous Certificate Authority (PCA)**: issues short-lived **pseudonymous** end-entity certificates to vehicles. A PCA knows only that a device holds a valid enrolment credential, not its real-world identity.
- **Enrolment CA**: issues a long-lived **enrolment credential** to a vehicle at manufacture time. The enrolment credential is used only to request pseudonymous certificates and is never transmitted in safety messages, preventing correlation by passive observers.
- **Linkage Authority (LA) and Misbehavior Authority (MA)**: provide a privacy-preserving revocation infrastructure. Each certificate carries a **linkage value** (LV) — a compact identifier derivable from a secret held by the LA. If a vehicle is reported for misbehaviour, the MA coordinates with the LA to generate CRL entries based on the LV without revealing the vehicle's real identity to any single entity (separation of knowledge).

The core operational mechanisms are:

- **Signed messages**: every safety-critical Wireless Short Message (WSM) is signed with ECDSA over P-256 or P-384, binding it to a pseudonymous certificate. The signature covers the message payload and a timestamp, preventing both tampering and replay.
- **Pseudonymous certificates**: vehicles hold a pool of short-lived certificates (validity period configurable; the US SCMS model provisions batches of approximately 1-week certificates). Vehicles rotate to a new pseudonym at privacy-sensitive moments (at intersections or after a fixed interval of use) to prevent long-term vehicle tracking via certificate continuity.
- **Certificate revocation**: compromised certificates are invalidated via CRL distribution. Distributing CRLs under intermittent VANET connectivity is an open engineering problem: proposed solutions include RSU-based CRL push at roadside,

Bloom-filter-compressed CRL entries embedded in beacon messages, and V2V epidemic broadcast of revocation notices.

The ETSI ITS security framework [2] complements IEEE 1609.2 with Authorisation Ticket (AT) mechanisms for Europe. An AT is a short-lived certificate authorising a specific ITS application permission. The ETSI model separates registration from authorisation: the Enrolment Authority (EA) and Authorisation Authority (AA) are distinct entities, so an AA cannot link an AT back to a vehicle's real identity, only confirm the vehicle holds a valid EA credential.

A comprehensive design for a Vehicular PKI (VPKI) reconciling privacy, revocability, and operational scalability is presented by Papadimitratos et al. [25]. The design influenced both the US Security Credential Management System (SCMS) and the European C-ITS credential management system.

2.4.3 Pseudonym Management

Pseudonymous certificates protect long-term privacy against passive observers correlating a vehicle across its journey via certificate identity. Effective pseudonym management requires balancing several competing concerns:

- **Pool size and provisioning**: a vehicle should hold enough certificates to rotate frequently without exhausting its supply between provisioning windows. The SCMS model provisions batches of certificates over the cellular network during periodic software updates.
- **Change frequency and timing**: fixed-interval rotation is weaker than random rotation, as an adversary correlating by timing can still link successive certificates. Changing pseudonym at busy intersections — where many vehicles change simultaneously, providing a crowd for k-anonymity — is more effective than changing on an empty road where a single vehicle changes and the change is trivially attributable.
- **Cross-layer linkability**: even with certificate rotation, higher-layer identifiers (IP addresses, application session tokens, Bluetooth beacons) can re-link a vehicle to its previous pseudonym. Full privacy requires coordinated re-randomisation of all identifiers simultaneously with the certificate change.
- **Linkability vs. revocability trade-off**: shorter certificate lifetimes and larger pools make tracking harder, but also make it harder for the MA to revoke all certificates belonging to a misbehaving vehicle before it rotates to new, unrevoked certificates.

2.4.4 Broadcast Authentication

Heartbeat and beacon messages are inherently broadcast, which complicates authentication. A receiver cannot know in advance the sender's public key to verify an asymmetric signature, and distributing a pre-shared group key for HMAC creates difficult key revocation challenges.

Asymmetric signatures (ECDSA as in IEEE 1609.2) provide the strongest guarantees: a receiver that obtains the sender's certificate can verify immediately and non-interactively. However, verification is computationally expensive relative to a heartbeat period. On embedded hardware, ECDSA-P256 verification takes on the order of 1 ms; at vehicular densities of hundreds of beacons per second from many neighbours, this load becomes significant. Certificate transmission overhead (a P-256 certificate is approximately 250 bytes) also inflates beacon size substantially.

TESLA (Timed Efficient Stream Loss-tolerant Authentication) [26] uses a **time-delayed key disclosure** mechanism to amortise the cost of asymmetric bootstrapping over many messages:

1. At bootstrap, the sender constructs a one-way hash chain of length n: starting from a random seed k_n, it computes $k_{n-1} = H(k_n), k_{n-2} = H(k_{n-1}), \cdots, k_0 = H(k_1)$. The sender commits to k_0 (the chain head) via a single asymmetrically signed bootstrapping message distributed to receivers.
2. To broadcast a message m_i in time epoch i, the sender uses k_i as a MAC key: $\text{tag}_i = \text{HMAC}(k_i, m_i)$. At the time of broadcast, k_i has not yet been disclosed to receivers.
3. In a subsequent epoch $j > i$, the sender broadcasts k_i. Receivers verify that $H^{j-i}(k_i) = k_{j-i}$ (confirming the key is on the pre-committed chain), then retroactively verify tag_i on buffered message m_i.

Security relies on receivers having an upper bound on sender–receiver clock skew: a receiver must be certain that k_i was not already known (i.e., already disclosed) at the time m_i was received. In vehicular networks, GPS provides a readily available, high-accuracy time reference satisfying this requirement. TESLA introduces an authentication latency of one disclosure interval (one heartbeat period typically), which must be weighed against the routing-stability benefit. An attacker who can inject packets in the disclosure window before verification can still cause temporary routing disruption, but cannot forge persistent authenticated messages.

HMAC [27] provides integrity and authenticity on unicast paths where a shared symmetric key has been established in advance. HMAC is appropriate for the OBU–server unicast channel once a DH session key is in place, but cannot be used for broadcast heartbeats where no pre-shared key exists between sender and arbitrary receivers.

2.4.5 Misbehavior Detection and Revocation

Certificate-based authentication prevents impersonation of honest nodes but cannot prevent a legitimately-credentialed node from behaving maliciously: dropping packets, injecting false position or routing information, or duplicating its identity across multiple physically distinct devices.

Watchdog monitoring [20] places nodes in promiscuous mode to overhear whether their next-hop relay actually forwards packets. A watchdog increments a failure counter each time it transmits a packet and does not observe the next hop forwarding it; above a threshold the next hop is removed from routing candidates. Limitations include collusion (two nodes confirm forwarding between each other without actually forwarding to the

destination) and full-duplex separation (the next hop may forward on a channel the watchdog cannot observe).

CONFIDANT (Cooperation of Nodes — Fairness in Dynamic Ad-hoc NeTworks) [21] extends watchdog monitoring with a reputation system. Nodes share ALARM messages reporting misbehaviour observations to a trust manager. The reputation score per neighbour is updated on each ALARM; nodes with a score below a threshold are excluded from routing. CONFIDANT is vulnerable to **slander attacks** where a malicious coalition fabricates ALARMs against honest nodes; weighting ALARMs by the reputation of their source partially mitigates this at the cost of a bootstrapping problem (new nodes have unknown reputations).

Certificate revocation closes the loop at the PKI level: the MA collects misbehavior evidence from the field, adjudicates, and instructs the LA to generate CRL entries for the offending vehicle's current and future pseudonymous certificates. A persistent misbehaver can thus be permanently excluded from the network even as it rotates pseudonyms. The challenge is distributing revocation information quickly enough that the window between detection and effective exclusion does not allow significant damage.

2.4.6 Cryptography Primer

A brief summary of the cryptographic primitives used in this project:

- **X25519 (Curve25519 Diffie–Hellman)** [28]: an elliptic-curve DH key exchange over the Montgomery curve Curve25519. Each party generates a 32-byte scalar private key a and computes the public key $A = a \cdot G$, where G is the curve base point. Both parties compute the shared secret $S = a \cdot B = b \cdot A$. The scalar multiplication is constant-time and provides 128-bit security.
- **HKDF (HMAC-based Key Derivation Function)** [29]: the two-step Extract-then-Expand construction. HKDF-Extract(salt, IKM) = HMAC(salt, IKM) produces a uniform pseudorandom key (PRK); HKDF-Expand(PRK, info, L) uses repeated HMAC applications to produce L output bytes, domain-separated by the `info` string. This ensures that the raw shared secret from X25519 is stretched and domain-separated into symmetric keys of the required length.
- **AEAD (Authenticated Encryption with Associated Data)**: AEAD constructions (AES-GCM, ChaCha20-Poly1305) provide both confidentiality and integrity. Encryption outputs ciphertext plus a 16-byte authentication tag computed over both the ciphertext and optional associated data; decryption rejects the plaintext if the tag does not verify. Nonce uniqueness is required: this implementation generates a 12-byte nonce from the OS CSPRNG for each encryption.
- **Ed25519 signatures** [30]: an EdDSA scheme over Edwards25519. A signing key produces a deterministic 64-byte signature over a message; verification uses the corresponding verifying (public) key. Ed25519 is used optionally in this work to authenticate DH handshake messages, binding them to node identities and preventing active man-in-the-middle substitution.

- **ML-KEM-768 (Module Lattice Key Encapsulation Mechanism)** [31]: a quantum-resistant Key Encapsulation Mechanism standardised as NIST FIPS 203. Unlike DH, it does not require both parties to contribute randomness to the shared secret; instead, one party (the encapsulator) generates the secret and transmits it encrypted under the other's public encapsulation key. The encapsulation key is 1184 bytes, the ciphertext 1088 bytes, and the shared secret 32 bytes. Its security rests on the hardness of the Module Learning With Errors (MLWE) problem rather than on discrete logarithms, making it resistant to Shor's algorithm [32] (see Section 2.4.7).
- **ML-DSA-65 (Module Lattice Digital Signature Algorithm)** [33]: a quantum-resistant signature scheme standardised as NIST FIPS 204, used as a drop-in replacement for Ed25519 when quantum resistance is required. The verifying key is 1952 bytes and the signature 3309 bytes. Like ML-KEM-768 it is based on MLWE and is not vulnerable to known quantum attacks (see Section 2.4.7).

These primitives compose as follows: X25519 or ML-KEM-768 produces a shared secret; HKDF derives symmetric keys; AEAD encrypts application payloads; Ed25519 or ML-DSA-65 (when enabled) signs handshake messages to authenticate the key exchange endpoints. The classical pair (X25519 + Ed25519) and the quantum-resistant pair (ML-KEM-768 + ML-DSA-65) are independently configurable.

2.4.7 Post-Quantum Cryptography

2.4.7.1 The Quantum Threat

Shor [32] demonstrated in 1994 that a sufficiently large quantum computer can solve the integer factorisation and discrete logarithm problems in polynomial time. This breaks all widely deployed asymmetric cryptography whose security relies on those problems: RSA, Diffie-Hellman over finite fields, and all elliptic-curve variants including X25519 (ECDH) and Ed25519 (ECDSA/EdDSA).

The practical timeline for cryptographically relevant quantum computers remains uncertain, but the **harvest now, decrypt later** (HNDL) attack is a present threat: an adversary with sufficient storage can capture encrypted traffic today and decrypt it retroactively once quantum hardware matures. For long-lived infrastructure such as vehicular network nodes — which may remain in service for a decade or more — the HNDL window is a genuine concern, motivating quantum-resistant key exchange now rather than at the point of quantum availability. Grover's algorithm provides a quadratic speedup for brute-force search, halving the effective security of symmetric ciphers; AES-256 retains 128-bit post-quantum security and is unaffected by this work.

2.4.7.2 NIST Post-Quantum Cryptography Standardisation

In 2016 NIST launched an open competition to standardise post-quantum cryptographic algorithms. After three evaluation rounds involving the global cryptographic community, NIST selected and standardised four algorithms in 2024:

- **FIPS 203 — ML-KEM** [31]: key encapsulation (based on CRYSTALS-Kyber)
- **FIPS 204 — ML-DSA** [33]: digital signatures (based on CRYSTALS-Dilithium)
- **FIPS 205 — SLH-DSA**: stateless hash-based signatures (based on SPHINCS+)

- **FIPS 206 — FN-DSA**: fast lattice-based signatures (based on FALCON)

vigilant-parakeet implements ML-KEM-768 (FIPS 203) and ML-DSA-65 (FIPS 204), the NIST Security Level 3 parameter sets of the two primary lattice-based standards. NIST Security Level 3 targets security equivalent to AES-192, meaning no known classical or quantum algorithm can break the scheme faster than exhaustive search of a 192-bit key space.

2.4.7.3 Module Learning With Errors

Both ML-KEM and ML-DSA derive their security from the **Module Learning With Errors** (MLWE) problem, a structured variant of the Learning With Errors (LWE) problem introduced by Regev [34]. Given a polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$ and a prime modulus q, the MLWE problem is:

Given a random matrix $\boldsymbol{A} \in R_q^{k \times k}$, a secret vector $\boldsymbol{s} \in R_q^k$ with small coefficients, and an error vector $\boldsymbol{e} \in R_q^k$ with small coefficients, **distinguish** $\boldsymbol{A}, \boldsymbol{As} + \boldsymbol{e}$ from $\boldsymbol{A}, \boldsymbol{u}$ where $\boldsymbol{u}$ is uniformly random.

For ML-KEM-768 and ML-DSA-65, $k = 3$ (the "768" designates the total ring dimension $k \cdot n = 768$). No polynomial-time classical or quantum algorithm is known for MLWE; the best known algorithms require sub-exponential time even on quantum hardware. The reduction from MLWE to ring-LWE and standard LWE is well-studied, providing a firm theoretical foundation for the security claims.

2.4.7.4 ML-KEM-768: Key Encapsulation Mechanism

A KEM differs from Diffie-Hellman in a fundamental way: it is **asymmetric in the production of the shared secret**. The holder of the encapsulation key (analogous to a public key) can only receive a secret; only the holder of the decapsulation key can recover it. There is no joint computation — the encapsulator generates the secret unilaterally and transmits it encrypted.

The ML-KEM-768 key sizes are governed by the ring dimension and modulus:

Object	Size	Role
Encapsulation key (public)	1184 B	Distributed to the party that will encapsulate secrets.
Decapsulation key seed	64 B	Stored securely by the key holder; full decapsulation key is derived on demand.
Ciphertext	1088 B	Transmitted from encapsulator to decapsulator; contains the encrypted shared secret.
Shared secret	32 B	Recovered by both parties; used as input to HKDF for session key derivation.

Table 3 — ML-KEM-768 key and ciphertext sizes

The encapsulation operation is $\mathsf{Encaps}(\mathsf{ek}) \to (\mathsf{ct}, \mathsf{ss})$: given the encapsulation key, it draws random coins, computes a ciphertext, and outputs a shared secret. Decapsulation is $\mathsf{Decaps}(\mathsf{dk}, \mathsf{ct}) \to \mathsf{ss}$: given the decapsulation key and the ciphertext, it recovers the

same shared secret. ML-KEM provides IND-CCA2 security (indistinguishability under adaptive chosen ciphertext attacks) under the MLWE assumption.

2.4.7.5 ML-DSA-65: Lattice-Based Digital Signatures

ML-DSA (formerly CRYSTALS-Dilithium) is a Fiat-Shamir with Aborts signature scheme. Signing generates a candidate response vector and rejects it if it leaks information about the signing key, repeating until a valid response is found. This abort-and-retry mechanism ensures that the signature statistically hides the secret key.

Object	Size	Role
Signing key seed	32 B	Compact representation; full signing key is derived on demand.
Verifying key	1952 B	Distributed to verifiers; pre-registered in peer allowlists for PKI mode.
Signature	3309 B	Carried in-band in the signed key exchange extension.

Table 4 — ML-DSA-65 key and signature sizes

ML-DSA-65 provides existential unforgeability under chosen message attacks (EUF-CMA) under the MLWE and MSIS (Module Short Integer Solution) assumptions, both of which are conjectured to be hard for quantum computers.

2.4.7.6 Size Overhead Compared to Classical Algorithms

The quantum-resistant algorithms carry significantly larger key material and signatures than their classical counterparts. Table 5 summarises the size overhead, which directly affects the on-wire message sizes for key exchange and authentication:

Algorithm	Role	Key / encap material	Sig / ciphertext
X25519	ECDH	32 B public key	—
Ed25519	Signature	32 B verifying key	64 B
ML-KEM-768	KEM	1184 B encap key	1088 B ciphertext
ML-DSA-65	Signature	1952 B verifying key	3309 B

Table 5 — Classical versus post-quantum algorithm sizes

For VANET applications, the larger sizes have two consequences. First, a fully signed ML-KEM-768 + ML-DSA-65 key exchange message can reach approximately 6.5 KB (1197 B base + 5266 B signed extension), requiring jumbo-frame support in the underlying transport — vigilant-parakeet sets `PACKET_BUFFER_SIZE = 9000` bytes to accommodate this. Second, the key exchange latency budget must account for the larger message traversal time across the wireless link. Critically, the overhead applies only to the session setup handshake; subsequent encrypted data frames carry only the 28-byte AEAD header (12-byte nonce + 16-byte tag) regardless of which key exchange algorithm was used.

2.4.8 Data Confidentiality

Early vehicular network designs prioritised availability and integrity — safety messages must arrive and must not be modified — treating payload confidentiality as secondary. As VANETs expand to carry non-safety traffic (internet access, toll transactions, content delivery), protecting payload data from relay nodes and eavesdroppers becomes significant.

The standard approach is **end-to-end encryption** at the OBU–server boundary: intermediate relay nodes (other OBUs, RSUs) forward ciphertext opaquely and never hold session keys. The central challenge in the VANET context is completing a key exchange within the contact window an OBU may have with a given RSU. Two-message Diffie-Hellman handshakes [28] are lightweight enough to complete in a single RSU contact and produce a forward-secret session key without requiring certificate infrastructure. Raya and Hubaux [35] analyse the broader trade-offs between pseudonymous certificate-based approaches and lightweight alternatives for vehicular security, concluding that lightweight DH-based schemes are more practical under the connectivity constraints of vehicular networks.

The long operational lifetime of vehicular infrastructure introduces an additional dimension: the **harvest now, decrypt later** threat. An adversary who records today's encrypted VANET traffic can store it and attempt decryption once quantum hardware capable of running Shor's algorithm [32] becomes available. For traffic whose confidentiality must be preserved beyond the expected quantum horizon, key exchange algorithms resistant to quantum attacks — such as ML-KEM-768 [31] — are necessary even before large-scale quantum computers exist.

2.4.9 Replay Window Mechanism

The **sliding receive window** is the standard technique for replay protection in authenticated packet protocols, used in IPsec AH [36] and IKEv2, and implemented in vigilant-parakeet's `ReplayWindow` for HeartbeatReply messages. The mechanism warrants a brief treatment here because the vigilant-parakeet implementation follows the IPsec design closely, including a non-obvious extension for window-poisoning prevention.

The core state is a pair `(last_seq: u32, window: u64)`:

- `last_seq` is the highest accepted sequence number seen so far.
- `window` is a 64-bit bitmask: bit i represents whether sequence number `last_seq - i` has been accepted (`1` = accepted; `0` = not yet seen).

A received packet with sequence number `seq` is processed as follows:

1. If `seq > last_seq + 1`: the packet is too far ahead. It is accepted; the bitmask is left-shifted by `seq - last_seq` (clearing bits that shifted off the low end), `last_seq` is updated to `seq`, and bit 0 is set.
2. If `seq == last_seq + 1`: the packet is the expected next sequence number. Accept, update `last_seq`, set bit 0.
3. If `last_seq - 64 < seq <= last_seq` (within the window): check bit `last_seq - seq`. If the bit is already set, the packet is a duplicate: drop. If the bit is clear, accept and set it.

4. If `seq <= last_seq - 64` (beyond the left edge of the window): drop. The packet is too old to verify uniqueness.

The bitmask efficiently represents the acceptance state of the last 64 sequence numbers in a single 8-byte integer. IPsec AH specifies a mandatory minimum window size of 32 and recommends 64; vigilant-parakeet uses 64.

The **window-poisoning attack** is a subtle vulnerability in naive implementations: an attacker forges a HeartbeatReply with `seq = u32::MAX` (or any value far above the current `last_seq`). The window shifts by `u32::MAX - last_seq`, advancing `last_seq` to `u32::MAX`. All subsequent legitimate replies with small sequence numbers (including `last_seq + 1` after wrap) now fall entirely outside the window and are dropped as "too old." The RSU's routing table stops updating for that sender.

IPsec counters this by rejecting packets whose sequence number is implausibly large relative to the expected next sequence number. vigilant-parakeet's defence is different and specific to the heartbeat context: the RSU maintains a ring buffer of the last N sequence numbers it **sent** in its own Heartbeats. Before calling `check_and_update()` on the replay window, the RSU verifies that the received `seq` appears in its sent-heartbeat ring. A forged large sequence number that was never sent is rejected at this pre-check step, before it can poison the window.

2.4.10 The Reference Security Model

The paper by [1] specifically analyses Layer-3 security threats in vehicular networks, with a focus on routing manipulation attacks in the OBU/RSU architecture. A central finding is that an adversary controlling one or more intermediate nodes can selectively drop, replay, or modify heartbeat messages to poison routing tables across a wide area — an attack that is efficient precisely because heartbeats carry no authentication. The routing protocol implemented in vigilant-parakeet is designed to reproduce the network model described in that work and serve as a platform for studying both the attacks and the defences proposed therein.

2.5 Simulation Approaches

2.5.1 Network Simulators

Traditional network simulators — ns-3 [37], OMNET++ [38], and SUMO [39] (for mobility) — provide rich, well-validated models but require significant configuration effort, are difficult to extend with custom Rust code, and do not run actual production-grade node logic.

ns-3 is a discrete-event network simulator implemented in C++ with a Python scripting API. It provides detailed models of 802.11p (WAVE), IEEE 1609.x, LTE, and many other protocols. Vehicular scenarios are typically coupled with SUMO for realistic mobility traces via the TraCI interface. ns-3 simulates a synthetic reimplementation of the protocol

stack on a single simulated host, not the actual binary that would be deployed on hardware.

OMNeT++ with VEINS provides an analogous SUMO coupling for vehicular simulations. The VEINS framework implements the IEEE 802.11p PHY/MAC and the IEEE 1609.x WAVE stack in C++ within OMNeT++'s module system. Like ns-3, OMNeT++/VEINS is a model-based simulator: protocol behaviour is reimplemented from specification rather than running real node code.

The fundamental limitation of model-based simulators for this work is the **fidelity gap**: any difference between the C++ simulation model and the Rust production code is a potential source of incorrect simulation results. Bugs in the simulator do not manifest on hardware, and bugs fixed on hardware do not automatically flow into the simulator.

2.5.2 Container and Namespace-Based Simulation

Linux **network namespaces** provide lightweight OS-level isolation of the network stack. Each namespace has its own routing table, interface list, and firewall rules, but shares the host kernel. This allows the same node binary that would run on real hardware to execute in an isolated environment with controlled inter-namespace connectivity, eliminating the fidelity gap.

Tools such as Mininet [40] popularised this approach for software-defined networking research. vigilant-parakeet applies the same technique to vehicular nodes: each OBU, RSU, and server executes in its own namespace with TUN/TAP virtual interfaces providing L2 connectivity. A userspace channel model applies configurable latency, packet loss, and jitter to inter-node links without requiring kernel `tc-netem` rules — all link emulation is handled in Rust async tasks, making the simulation fully portable within the Linux kernel.

A closely related contemporary work is **Nextmini** [41], a general-purpose network emulation testbed targeting datacenter and cloud networking research. Like vigilant-parakeet, Nextmini is implemented in Rust with the Tokio async runtime, uses Linux network namespaces for node isolation, and exposes TUN interfaces so that arbitrary application binaries can run inside the emulated network without modification. Its dataplane adopts an **actor model** in which each node's forwarding logic runs as one or more Tokio tasks communicating via MPSC channels, avoiding shared mutable state across the dataplane.

Nextmini's architecture diverges from vigilant-parakeet in several respects that reflect its different target domain:

- **Control plane**: Nextmini separates control from data through a centralised Rust web server controller backed by a PostgreSQL database. Control-plane algorithms are expressed as Python scripts that query and update the database; real-time PostgreSQL triggers notify the controller when routing decisions must be pushed to the dataplane. vigilant-parakeet's routing is fully distributed — RSUs emit heartbeats, OBUs build routing tables independently with no central coordinator.
- **Inter-node connectivity**: Nextmini connects dataplane nodes via persistent TCP or QUIC connections, optimised for high bulk throughput (up to 132 Gbps in its

`max` mode using the Linux `splice` system call). vigilant-parakeet uses TUN/TAP virtual interfaces with a userspace channel layer that imposes per-link latency, loss, and jitter — a model suited to studying routing under impaired wireless conditions rather than maximising throughput.

- **Scale and target**: Nextmini targets up to 10,000 dataplane nodes on a single machine (approximately 4.5 MB per node) and is designed for large-scale datacenter experiments including distributed ML training. vigilant-parakeet targets small-to-medium vehicular topologies (tens of nodes) where the focus is routing protocol correctness and security rather than forwarding throughput.
- **Channel impairment model**: Nextmini does not model wireless channel conditions — latency and loss are not first-class concepts in its topology description. vigilant-parakeet's per-link `Channel` objects with configurable latency, loss, and jitter are central to its purpose of evaluating routing behaviour under vehicular channel conditions.
- **Domain specificity**: Nextmini is domain-agnostic: it runs any application workload (web servers, ML training jobs) and any routing algorithm expressible in Python. vigilant-parakeet encodes VANET-specific behaviour (OBU/RSU roles, heartbeat-based route building, DH key exchange, N-best candidate caching) directly in its node library crates.

In summary, Nextmini and vigilant-parakeet occupy different positions on the spectrum between general-purpose emulation infrastructure and domain-specific simulation: Nextmini provides a highly scalable, flexible substrate for datacenter networking research with no built-in protocol assumptions; vigilant-parakeet sacrifices generality to provide first-class support for vehicular routing dynamics, wireless channel impairment, and the OBU–RSU security architecture that motivates this work.

The key advantage that both share over model-based simulators is **fidelity**: the same binary runs in simulation and in production. Bugs manifest in both environments; fixes in production automatically flow into the simulator.

2.5.3 Rust for Systems Programming

Rust [42] provides memory safety without garbage collection, making it well-suited for implementing network protocol logic that must be both correct and efficient. The Tokio [43] asynchronous runtime allows many nodes to run concurrently on a shared thread pool, enabling dense simulations (dozens of nodes) on commodity hardware without per-node OS threads. The async model also makes latency injection natural: a sleeping `tokio::time::sleep` future correctly models a link delay without blocking any OS thread.

2.5.3.1 Ownership, Borrowing, and Memory Safety

Rust's central innovation is its **ownership** discipline enforced at compile time by the borrow checker. Every value has a single owner; when the owner goes out of scope, the value is dropped (its destructor runs and memory is freed) without a garbage collector. Shared read-only access is provided by **shared references** (`&T`), exclusive mutable access by **mutable references** (`&mut T`). The borrow checker ensures that at most one mutable

reference or any number of shared references to a value can exist at any time — the "aliasing XOR mutability" invariant.

For network protocol code, this eliminates:

- **Buffer overflows**: slice indexing is bounds-checked; out-of-bounds access panics or returns an `Option::None` rather than reading adjacent memory.
- **Use-after-free**: the owner of a resource holds the sole live reference; all borrowed references are statically proven to not outlive the owner.
- **Data races**: mutable access to shared data requires mutual exclusion (`Mutex<T>`, `RwLock<T>`); the compiler rejects code that shares `&mut T` across threads without synchronisation. The `Send` and `Sync` marker traits enforce this at the type level.

For a VANET simulator where multiple nodes share routing state via `Arc<RwLock<RoutingTable>>`, these guarantees mean that deadlocks and data races are the programmer's last concern rather than a constant source of debugging. The integration test suite runs under address sanitiser in CI (`RUSTFLAGS="-Z sanitizer=address"`) with no false positives expected, because the borrow checker has already excluded the bugs that sanitisers catch.

2.5.3.2 Async/Await and the Tokio Runtime

Tokio implements a cooperative, multi-threaded async executor. An `async fn` in Rust desugars to a state machine implementing the `Future` trait: each `.await` point is a yield point at which the task can be suspended and another task scheduled. The key properties relevant to vigilant-parakeet are:

M:N thread model. Tokio maps many Tokio tasks (logical coroutines) onto a smaller, configurable pool of OS threads (default: one per logical CPU). On the evaluation machine (24 threads), dozens of node tasks share 24 OS threads without per-task OS-thread overhead. The memory overhead of a Tokio task is the size of its captured state (typically a few hundred bytes for a simple I/O loop) rather than the 2–8 MB stack of an OS thread.

I/O without blocking. TUN read operations are wrapped with `tokio::io::AsyncReadExt`; when no frame is available, the task yields, freeing its OS thread for other tasks. A single OS thread can multiplex reads from many TUN interfaces simultaneously.

Mocked time. `tokio::time::pause()` stops the Tokio time source; `tokio::time::advance(duration)` advances it by the given amount without wall-clock delay. This allows integration tests to simulate 12 hours of re-keying cycles in milliseconds, and to test latency-measurement logic with precise, reproducible timing.

Work stealing. Tokio uses a work-stealing scheduler: if one OS thread runs out of tasks while another has a backlog, it steals tasks from the other's queue. This makes throughput robust to uneven task distribution without manual thread assignment.

2.5.3.3 Compile-Time Zero-Cost Abstractions

Rust's generics and trait system enable abstractions that have zero runtime cost (no virtual dispatch, no heap allocation) when the concrete type is known at compile time. The `Tun` and `Device` traits in `common` are monomorphised: in production code, `create<T: Tun>(...)` is instantiated with the real `TokioTun` or `tun_tap::Iface` types, and the compiler inlines all trait method calls. In test code, the same generic is instantiated with the shim

implementation backed by Tokio channels. The result is that the test and production code paths are structurally identical — the compiler can verify both — but generate completely different machine code appropriate to each context.

3. Related Work

This chapter surveys work related to vigilant-parakeet across three dimensions: simulation and emulation platforms for vehicular and ad-hoc networks, routing protocol implementations, and security frameworks with particular attention to post-quantum migration challenges in V2X environments. The chapter concludes with a consolidated comparison that positions vigilant-parakeet within this landscape.

3.1 VANET Simulation and Emulation Platforms

A central design choice for any VANET research tool is the trade-off between **model fidelity** (how faithfully the simulator reproduces real hardware behaviour) and **execution fidelity** (whether the actual production code runs, or a synthetic reimplementation). This distinction drives almost all architectural differences between the tools reviewed here.

3.1.1 Coupled Mobility and Network Simulators

The dominant research methodology for VANET evaluation combines a **road traffic simulator** with a **network event simulator**, coupling them so that vehicle positions and speeds from the traffic side drive node placement in the network side.

VEINS (Vehicles in Network Simulation) [44] is the most widely used vehicular simulation framework. It couples OMNeT++ [38] with SUMO [39] via the TraCI (Traffic Control Interface) socket protocol. SUMO computes vehicle dynamics at each simulation step; the TraCI bridge pushes position and velocity updates to OMNeT++ nodes. VEINS implements the IEEE 802.11p (WAVE) physical and MAC layers in C++, using the Two-Ray interference model or the Nakagami-m fading model to compute received signal strength and packet error rates as a function of inter-vehicle distance, obstacle geometry, and mobility. The bidirectional coupling means that a VANET application can react to the simulated traffic environment and, if desired, influence traffic dynamics (e.g., a platoon controller adjusting vehicle spacing in response to a CACC message).

VEINS ships with modules for the full IEEE 1609.x WAVE stack, Cooperative Awareness Messages (CAMs), Decentralised Environmental Notification Messages (DENMs), and the V2X application layer defined in ETSI ITS [2]. This makes it well suited for evaluating standard-compliant V2X applications. Its limitation for this work is the fidelity gap: VANET routing and security logic must be reimplemented in C++, so any divergence between the OMNeT++ model and the production Rust implementation is a potential source of false results. VEINS also provides no means of running arbitrary compiled binaries inside simulated nodes — only code written against its C++ framework executes.

VSimRTI [45] (V2X Simulation Runtime Infrastructure), developed at the German Aerospace Centre (DLR), takes a federation approach: it defines a common coupling API that allows multiple simulators to be interconnected — SUMO for road traffic, ns-3 [37] or OMNeT++ for the VANET, and custom application simulators. VSimRTI acts as the middleware, synchronising simulation time and routing events between federated components. This enables researchers to swap out any individual component without restructuring the full simulation setup, which is valuable for comparative studies. The underlying limitation is the same as VEINS: application logic must be implemented within the simulation framework rather than running real executables.

TraNS [46] (Traffic and Network Simulation Environment) was an earlier coupling of ns-2 and SUMO, predating VEINS. It established the now-standard pattern of bi-directional simulation coupling for VANETs, and while ns-2 has largely been superseded by ns-3 in new research, TraNS demonstrated that mobility-driven network simulation was viable and influenced the design of subsequent coupling frameworks.

ns-3 [37] provides a Wave/80211p module that can be used standalone (with synthetic mobility models such as the GaussMarkov or RandomWaypoint models) or coupled with SUMO via TraCI. ns-3′s protocol stack includes models of the WAVE MAC (EDCA, channel coordination, wave service advertisement), the DSRC physical layer, and IPv6/GeoNetworking. All of this is a model-based reimplementation.

3.1.2 Emulation-Based Approaches

Unlike model-based simulators, emulation-based tools run real application code in an isolated environment, eliminating the fidelity gap at the cost of reduced scalability and determinism.

Mininet [40] pioneered lightweight network emulation using Linux network namespaces. Each Mininet host is a namespace with virtual Ethernet links interconnected by an in-kernel Open vSwitch; latency and bandwidth limits are applied with kernel traffic control (`tc-netem`). Mininet was designed for software-defined networking research: its default topology abstraction and Python API assume OpenFlow-controlled Ethernet switches, and its wireless support is limited without extensions.

Mininet-WiFi [47] extends Mininet with IEEE 802.11 wireless emulation using the `mac80211_hwsim` virtual radio driver in the Linux kernel. Each node's wireless interface is backed by a `hwsim` radio, which passes frames through the kernel 802.11 stack. Signal propagation is modelled using adjustable path-loss models (log-distance, two-ray, ITU) applied in userspace. Mininet-WiFi is closer to the vehicular use case than plain Mininet, but it is oriented toward WiFi infrastructure and mesh scenarios; vehicular-speed mobility and 802.11p WAVE signalling are not first-class concepts.

GNS3 [48] (Graphical Network Simulator-3) is a heavyweight emulation platform that runs node images in QEMU virtual machines or Docker containers. Full routing stack binaries (Cisco IOS, Juniper JunOS, FRRouting) execute natively, giving extremely high execution fidelity for production routers. The cost is resource overhead: each node requires a separate VM or container with its full OS image. GNS3 is well suited for realistic

networking lab scenarios but impractical for dense vehicular topologies where tens of nodes must run concurrently on a single machine.

Kathara [49] replaces GNS3′s VM model with Docker containers, reducing per-node overhead while retaining execution fidelity. A Kathara network is specified as a directory of container images and interface configuration files, making topologies reproducible and version-controllable. Kathara has been used for network function virtualisation and SDN research; its vehicular application is limited by the same absence of wireless channel modelling present in all container-based emulators.

Shadow [50] takes a distinct approach: rather than running processes in separate containers or namespaces, it uses a deterministic discrete-event execution model. Shadow intercepts system calls (via `preload` or `ptrace`) and replaces I/O with a simulated network that advances according to a global event queue. The key benefit is **determinism**: given the same seed, every execution of a Shadow simulation produces identical results, enabling reproducible network experiments at scale. Shadow was developed for Tor anonymity network research and has been used to evaluate Tor at full scale (thousands of relays and clients) on a single machine. Its limitation for VANET research is that its network model does not support wireless channels, and the event-driven execution model makes it difficult to integrate with physical-layer simulation.

vigilant-parakeet occupies its own position in this landscape: like emulation-based tools, it runs real compiled node code (the same `obu_lib`, `rsu_lib`, and `server_lib` crates that would be deployed on real hardware); like model-based simulators, it applies a configurable channel model (latency, loss, jitter) to each link. The channel model is simpler than VEINS' IEEE 802.11p simulation — no signal attenuation, no collision, no spatial reuse — but this simplicity is appropriate for studying routing convergence and security protocols, which depend on delivery statistics rather than PHY-level behaviour. The core design principle is that any bug fixed in the production node code is automatically fixed in simulation, without a separate maintenance burden.

Platform	**Runs real code**	**Wireless channel**	**Deterministic**	**Primary domain**
VEINS/OMNeT++	No	Yes (802.11p model)	Yes	V2X standard compliance
ns-3 + SUMO	No	Yes (802.11p model)	Yes	Protocol stack research
VSimRTI	No	Yes (federated)	Partial	Multi-simulator coupling
Mininet	Yes	No	No	SDN / OpenFlow research
Mininet-WiFi	Yes	Yes (hwsim, 802.11)	No	WiFi / mesh research
GNS3	Yes	No	No	Production router testing

Kathara	Yes	No	No	NFV / SDN research
Shadow	Yes	No	Yes	Large-scale Tor / overlay
Nextmini [41]	Yes	No	No	Datacenter / ML training
vigilant-parakeet	**Yes**	**Config-urable**	No	**VANET routing + security**

Table 6 — Comparison of network simulation and emulation platforms

3.2 Routing Protocol Implementations

Published VANET routing protocol research is predominantly model-based: the routing algorithm is evaluated in VEINS or ns-3, not in a running Linux implementation. Production-grade, Linux-native implementations of ad-hoc routing protocols exist but are generally not vehicular-specific.

B.A.T.M.A.N.-Advanced (batman-adv) [51] is a mesh routing protocol implemented as a Linux kernel module (`net/batman-adv`), operating at Layer 2. Batman-adv distributes routing responsibility across the network: each node periodically floods **Originator Messages** (OGMs) carrying a transmission quality metric derived from observed forwarding success rates. A node selects the neighbour through which it has historically received the most OGMs from a given originator as its best transmitter (TQ: Transmission Quality) toward that originator. Batman-adv is deployed in community mesh networks (Freifunk, Wireless Battle of the Mesh) and shares vigilant-parakeet's principle of emitting periodic control beacons for route building, but targets a flat Layer-2 forwarding model rather than the hierarchical OBU/RSU V2I architecture.

AODV-UU [52] (AODV for Linux, from Uppsala University) is a userspace daemon implementation of AODV [13] for Linux. It intercepts packets via Netfilter hooks, initiates route discovery as needed, and installs kernel routing table entries for discovered routes. AODV-UU was used for vehicular research before VEINS-based simulation became dominant. Its limitation in vehicular contexts — route discovery latency exceeding RSU contact windows — is the same limitation identified for AODV in Section 2.

OLSRd is the IETF reference implementation of OLSR [12], used in community wireless mesh networks. Like batman-adv, it operates in a relatively stable pedestrian-speed mesh context and is not designed for vehicular speeds.

The heartbeat-based routing protocol implemented in vigilant-parakeet differs from all of the above in that it is **infrastructure-anchored**: the RSU is the routing destination (not an arbitrary peer), and route building is driven by RSU-emitted beacons rather than node-initiated floods. This model is closer to the Internet's distance-vector model applied to a single-tier hierarchy (OBUs routing **toward** a fixed gateway RSU) than to flat mesh routing. No production Linux implementation of this specific model — heartbeat-driven upstream candidate selection with N-best caching, composite latency/hop-count metric, and integrated DH key exchange — was found in the existing literature.

3.3 VANET Security Frameworks

3.3.1 Deployed Credential Management Systems

The US **Security Credential Management System** (SCMS) is the operational V2X PKI infrastructure deployed by the US Department of Transportation and vehicle manufacturers for connected vehicle programmes. Following the IEEE 1609.2 architecture described in Section 2.4.2, the SCMS issues and manages pseudonymous certificates for V2X message signing using ECDSA over P-256. It separates identity management (Enrolment Authority) from application authorisation (Policy Certificate Authority) and provides a privacy-preserving revocation mechanism via Linkage Authorities. The SCMS handles the trust establishment problem that vigilant-parakeet addresses for OBU–server key exchange: in vigilant-parakeet, trust is bootstrapped via Ed25519 or ML-DSA-65 signing keys provisioned at manufacture time (PKI mode) or via TOFU on first contact, rather than a full certificate hierarchy. This simplification is appropriate for a research prototype but would need to be replaced with a full credential management system in a production deployment.

The European counterpart is the **C-ITS Credential Management System** (CCMS), standardised under ETSI ITS [2], using Authorisation Tickets (ATs) and the ETSI ITS security header format rather than IEEE 1609.2 WSM headers. Both systems share the fundamental architecture of short-lived pseudonymous certificates, hierarchical PKI, and privacy-preserving revocation.

3.3.2 Research Prototypes and Security Analysis

[35] conducted one of the earliest comprehensive security analyses of VANETs, identifying the key threats — impersonation, bogus information, identity disclosure, denial of service — and proposing a certificate-based security architecture as the solution. Their analysis prefigured much of what became IEEE 1609.2, and their conclusion that lightweight DH-based session key establishment is more practical than certificate-based channel encryption under vehicular connectivity constraints directly motivates the session key architecture in vigilant-parakeet.

[1] specifically analyses Layer-3 routing attacks in the OBU/RSU architecture — the attack model most directly relevant to this work. That paper identifies routing table poisoning via crafted or replayed heartbeat messages as the primary control-plane threat, and proposes defences based on authenticated heartbeats and sequence-number freshness. vigilant-parakeet implements the replay-freshness defence (the `ReplayWindow` at RSUs) and provides the infrastructure for studying the remaining authentication mechanisms identified in that work as future work items.

[25] presents a comprehensive Vehicular PKI design reconciling privacy, revocability, and scalability — the design that most influenced the US SCMS and European CCMS. The pseudonym management challenges described in that paper (Section 2.4.2) are relevant to the TOFU vs. PKI design decision in vigilant-parakeet: TOFU is appropriate in scenarios

where PKI infrastructure is unavailable, but it lacks the revocability properties that make a full PKI necessary for production deployment.

3.3.3 Post-Quantum Security in Vehicular Networks

The post-quantum migration of V2X security is an active research and standardisation area. [53] (ETSI TR 103 619) analyses the quantum threat to deployed ICT systems, including vehicular communications, and identifies priority migration targets: key exchange (X25519 → ML-KEM) and digital signatures (ECDSA/Ed25519 → ML-DSA). The document notes that the vehicular context presents specific constraints: 802.11p channel access units have a maximum MPDU size of approximately 2 KB (limited by OFDM symbol count and channel access duration), which is incompatible with a fully signed ML-KEM-768 key exchange message (6.5 KB including ML-DSA-65 signature). Proposed mitigations include message fragmentation, compressed certificate chains, and hybrid classical+PQC schemes that add only the PQC component (e.g., an ML-DSA-65 co-signature) to existing ECDSA-signed messages.

[54] proposes a lattice-based V2X security scheme (LSSS) derived from CRYSTALS-Kyber and CRYSTALS-Dilithium (the pre-standardisation precursors to ML-KEM and ML-DSA). Their evaluation on constrained vehicular hardware (ARM Cortex-M4) reports key exchange operations taking on the order of tens of milliseconds — acceptable for session setup but non-trivial on the low-power embedded units deployed in existing vehicles. The authors also note the message size challenge: their scheme requires message fragmentation for 802.11p transport.

vigilant-parakeet addresses both concerns differently: it targets a **software simulator** rather than constrained hardware, so compute latency is not a first-order constraint; and the simulated transport layer is a TUN device with configurable MTU (default 1 400 bytes) rather than 802.11p, so frame size constraints can be relaxed by raising `PACKET_BUFFER_SIZE` to 9 000 bytes. This makes vigilant-parakeet a useful platform for evaluating post-quantum key exchange overhead in terms of handshake latency and session establishment time, without the PHY-layer fragmentation problem that complicates evaluation on 802.11p simulators.

3.4 Summary and Positioning

Table 7 summarises how vigilant-parakeet compares to the most relevant related systems across the dimensions most important to this work.

System / Work	**OBU/RSU model**	**Real code**	**PQ crypto**	**Routing security**
VEINS + OMNeT++	Yes	No	No	Via extensions
ns-3 WAVE	Partial	No	No	Research modules
Mininet-WiFi	No	Yes	No	No
SCMS / CCMS	Yes	N/A	Planned	Certificate-based

[35]	Yes	No	No	Analysis only
[1]	Yes	No	No	Analysis + design
[54]	No	No	Yes	Signature only
vigilant-parakeet	**Yes**	**Yes**	**Yes**	**Session key + replay window**

Table 7 — Positioning of vigilant-parakeet relative to related work

vigilant-parakeet is the only system in this survey that combines: execution of real compiled VANET node code inside an isolated network namespace; a configurable post-quantum cipher suite for OBU–server key exchange and handshake authentication; and a routing security mechanism (HeartbeatReply replay window) directly addressing the attack model of [1]. The trade-offs compared to VEINS are explicit: vigilant-parakeet provides no 802.11p PHY-layer simulation, no SUMO mobility coupling, and no standard-compliant V2X application stack. It sacrifices breadth of simulation coverage to achieve high fidelity for the specific combination of routing behaviour and cryptographic session establishment that motivates this work.

4. Architecture

This chapter describes the high-level design of vigilant-parakeet: the crate decomposition, the three-tier network model, and the key design decisions.

4.1 Workspace Overview

vigilant-parakeet is organised as a Rust Cargo workspace with ten crates (Figure 1). The central principle is strict separation of concerns: shared protocol code lives in `node_lib` and `common`; node-type behaviour is encapsulated in `obu_lib`, `rsu_lib`, and `server_lib`; orchestration and analysis tooling sit in `simulator` and `scripts_tools`; key generation is provided by the standalone `keygen` utility.

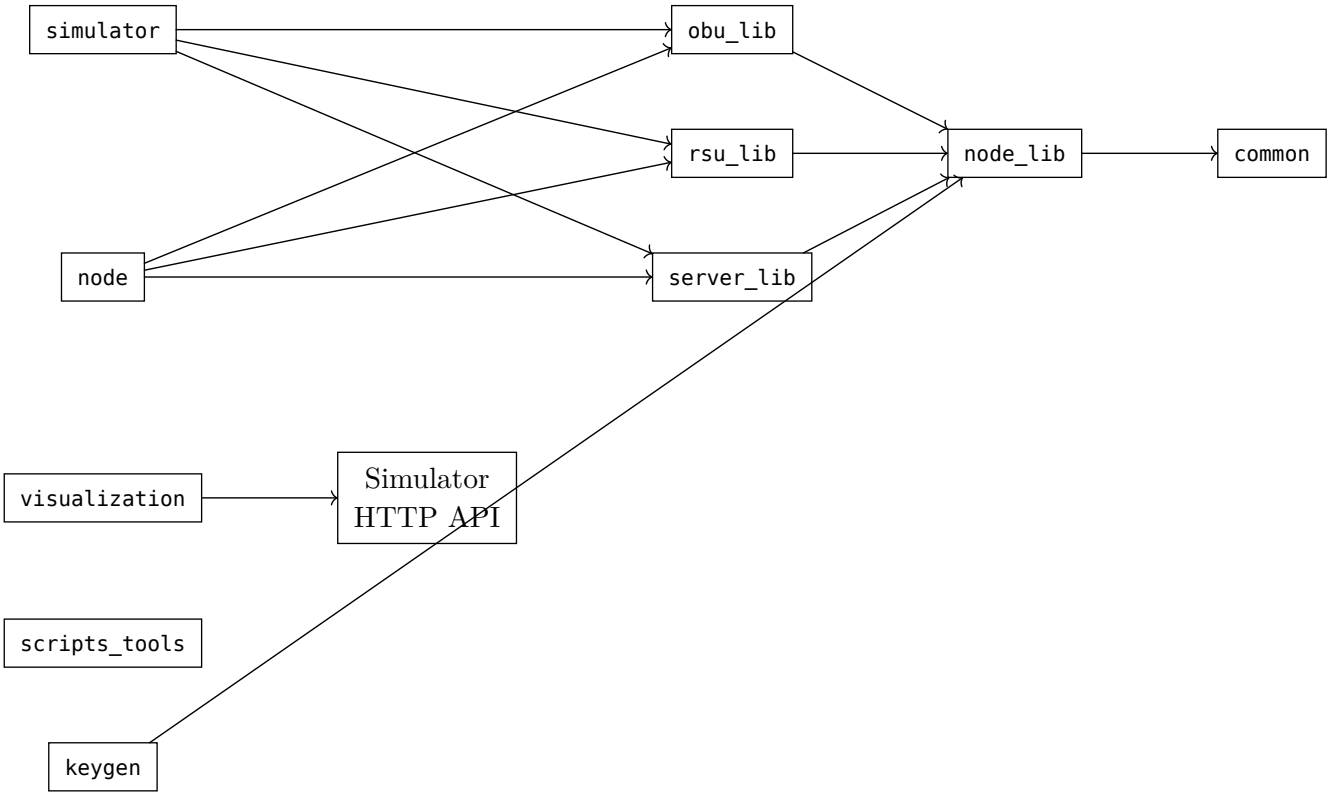

Figure 1 — Cargo workspace dependency graph

4.2 Three-Tier Network Model

The system implements a three-tier network that separates concerns between wireless VANET communication, infrastructure forwarding, and application-layer connectivity (Figure 2).

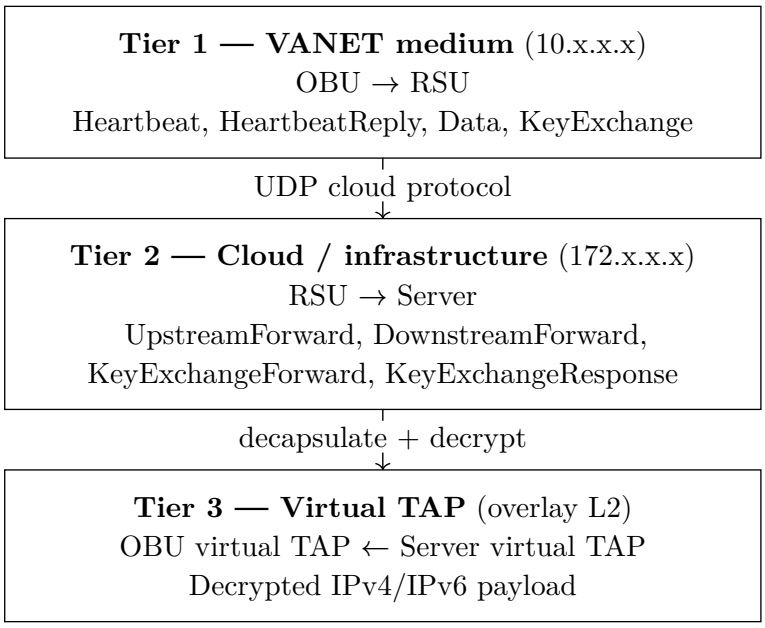

Figure 2 — Three-tier network architecture

- **Tier 1 (VANET)** is the wireless medium. OBUs and RSUs exchange routing control messages (Heartbeat, HeartbeatReply) and carry encrypted payload frames in the `Data` message type.
- **Tier 2 (cloud)** is the infrastructure link between RSUs and the server, implemented over UDP. RSUs are transparent relays: they wrap received VANET frames in `UpstreamForward` messages and send them to the server without inspecting or decrypting the payload.
- **Tier 3 (virtual TAP)** is an overlay L2 network shared between OBUs and the server. After decryption, the server injects plaintext frames into its virtual TAP device; the OBU reads decrypted frames from its own virtual TAP.

4.3 Crate Responsibilities

4.3.1 `common` — OS Abstractions

Provides platform-level building blocks:

- **`Tun` trait** with a `TokioTun` test shim for non-privileged integration testing.
- **`Device`** — wraps a network interface for send/receive.
- **`NetworkInterface`**, **`ChannelParameters`** — interface configuration helpers.
- **`stats`** — optional feature-gated atomic counters.

4.3.2 `node_lib` — Protocol Building Blocks

Cross-cutting protocol logic shared by all node types:

- **`messages/`** — wire encoders/decoders: `Heartbeat` (30 B: duration + id + hops + source), `HeartbeatReply` (36 B: + sender), `Data`, `KeyExchangeInit`, `KeyExchangeReply`, `PacketType`, `Message`. All fields are big-endian; `TryFrom<&[u8]>` and `Into<Vec<u8>>` are implemented for zero-copy deserialisation.
- **`crypto/`** — full configurable cipher suite: X25519 DH, HKDF, AES-256-GCM / AES-128-GCM / ChaCha20-Poly1305.
- **`data/`** — data-path helpers split by node type (`data::obu`, `data::rsu`).
- **`control/`** — shared routing utilities: `route`, `routing_utils`, `client_cache`, `ReplyType`.
- **`args/`** — `NodeType` enum (`Rsu | Obu | Server`) used by the node binary and simulator.
- **`buffer_pool`** — three-tier pool of reusable `BytesMut` buffers to reduce hot-path allocation: SMALL (256 B), MEDIUM (512 B), LARGE (1 500 B), with a capacity of 32 buffers per tier stored in a thread-local `Mutex<Vec<BytesMut>>`. A caller acquires the smallest tier that fits the requested length; on release the buffer is returned to the pool rather than dropped.
- **`control::ReplyType`** — discriminates how a processed frame should be delivered: `WireFlat` sends the frame out on the VANET device (tier 1), while `TapFlat` injects it into the virtual TAP interface (tier 3). This enum decouples the control-plane dispatch logic from the physical write path.
- **`Node` trait** — `trait Node: Send + Sync` implemented by `Obu` and `Rsu`. Exposes `as_any()` for runtime downcasting inside the simulator. `Server` is held directly as `Arc<Server>` and does not implement this trait.
- **`test_helpers/`** — `Hub` and `mk_shim_pair()` for non-privileged integration testing.

4.3.3 Internal Async Task Architecture

Each node type decomposes its work into a fixed set of Tokio tasks, spawned when the node starts. The tasks communicate via shared `Arc<RwLock<...>>` state and `tokio::sync` primitives (channels, notifications, cancellation tokens). Understanding this task structure is important for reasoning about concurrency properties and for diagnosing performance under load.

4.3.3.1 OBU Task Structure

An OBU spawns the following Tokio tasks:

VANET receive loop reads raw Ethernet frames from the `vanet` TUN interface, dispatches on message type (`Heartbeat` → routing update, `HeartbeatReply` → forward with `ForwardAction` check, `Data::Downstream` → decrypt + write to virtual TAP, `KeyExchangeReply` → complete DH exchange), and writes replies back to the VANET interface.

TAP transmit loop reads frames from the virtual TAP interface (application traffic destined for the server), encrypts with the established session key, and emits `Data::Upstream` on the VANET interface.

DH rekey timer sleeps for `dh_rekey_interval_ms`, then checks whether the session key is older than `dh_key_lifetime_ms`. If so, it calls `rekey_notify.notify_one()` to wake the rekey task.

DH rekey executor waits on `rekey_notify`; when notified, sends a `KeyExchangeInit` to the server via the current upstream and marks the key store as `Pending`. Also handles the `dh_reply_timeout_ms` timeout: if no `KeyExchangeReply` arrives within the timeout, it retries up to the configured maximum.

Admin console listener accepts TCP connections on `admin_port`, spawns a short-lived task per connection that reads line-oriented commands and replies.

All tasks share a `CancellationToken`; when the simulator sends a cancellation signal, each task observes the token at its next yield point and exits cleanly.

4.3.3.2 RSU Task Structure

An RSU spawns:

Heartbeat emitter sleeps for `hello_periodicity` ms, increments the sequence counter, serialises a `Heartbeat`, and writes it to the VANET broadcast address. Updates the `sent` history ring for replay-window validation.

VANET receive loop dispatches `HeartbeatReply` messages to the routing table updater and forwards `Data::Upstream` / `KeyExchangeForward` datagrams to the server via the cloud UDP socket.

Cloud receive loop reads from the cloud UDP socket; forwards `Data::Downstream` and `KeyExchangeResponse` back to the appropriate OBU on the VANET interface.

Admin console listener as for OBU.

The RSU has no DH or TAP tasks — it is a stateless relay at the session-key level.

4.3.4 `obu_lib` — OBU Node

Implements the full OBU control plane:

- Heartbeat reception, routing table management (1117-line `Routing`), N-best upstream caching and failover.
- DH key store (`DhKeyStore`): pending → established key lifecycle per server, with retry and expiry.
- TAP session: reads frames from the virtual TAP, encrypts them, and sends them upstream.
- **TCP admin interface** (`admin.rs`): interactive console for inspecting upstream routes, DH session state, and triggering manual re-key (see Section 5.5).
- Two network interfaces: `vanet` TAP (tier 1) and `virtual` TAP (tier 3).

Key `ObuParameters` defaults (from `obu_lib::args`):

Parameter	Default	Purpose
`hello_history`	`10`	Max heartbeat entries retained per RSU–sender pair in the routing table
`cached_candidates`	`3`	Number of N-best upstream candidates stored for fast failover
`dh_rekey_interval_ms`	`43 200 000`	Proactive re-key interval: 12 hours

`dh_key_lifetime_ms`	`86 400 000`	Maximum session key age before the OBU forces a new exchange: 24 hours
`dh_reply_timeout_ms`	`5 000`	Timeout waiting for a `KeyExchangeReply` before retrying
`mtu`	`1 400`	Maximum TAP frame size forwarded upstream

Table 8 — Key `ObuParameters` defaults

4.3.5 `rsu_lib` — RSU Node

Implements the RSU control plane:

- Periodic Heartbeat emission and HeartbeatReply processing.
- Client cache (`ClientCache`): tracks OBU VANET MAC → virtual MAC associations.
- **No virtual overlay TAP**: unlike OBUs and the server, RSUs have no decapsulated-traffic interface. They only have a `vanet` TAP (tier 1) and a `cloud` UDP socket (tier 2). They never decrypt or inspect OBU payload.
- Forwards VANET data opaquely to the server as `UpstreamForward` UDP messages.

4.3.6 `server_lib` — Server Node

The end-to-end trust anchor:

- Listens on a UDP `cloud` socket (tier 2) for RSU-forwarded messages.
- Owns a `virtual` TAP device (tier 3) for decapsulated application traffic.
- Manages per-OBU DH-derived keys; decrypts `UpstreamForward` payloads.
- Routes downstream traffic back to the correct RSU using an `obu_routes` table (`virtual TAP MAC → (VANET MAC, RSU UDP addr)`).
- RSU registration: `RegistrationMessage` keeps `RSU MAC → [OBU MACs]` up to date.
- The binary cloud protocol uses magic prefix `[0xAB, 0xCD]` and a 1-byte type discriminant (see Section 6 for wire formats).

4.3.7 `node` — Single-Node Binary

Three CLI subcommands: `node obu`, `node rsu`, `node server`. Each delegates directly to the corresponding library's `create()` function. This thin binary layer means the same library code used in integration tests is also the production entrypoint — there is no separate "simulation mode."

4.3.8 `simulator` — Multi-Node Orchestration

The simulator is the central orchestration layer. Its responsibilities are:

1. **Configuration loading**: reads a YAML file describing node types, per-node config paths, and a topology matrix specifying which nodes are adjacent and with what baseline channel parameters.
2. **Namespace provisioning**: creates one Linux network namespace per node using `netns_rs::NetNs::new("sim_ns_<name>")`. Each namespace has an independent network

stack; nodes cannot communicate except through the TUN/TAP interfaces the simulator creates.

3. **Interface and node creation**: delegates to the `node_factory` module (`simulator::node_factory`), which creates the correct set of virtual interfaces inside the namespace context and instantiates the appropriate library (`obu_lib`, `rsu_lib`, `server_lib`).
4. **Channel management**: each directed link is represented by a `Channel` object that applies configurable latency, loss, and jitter entirely in userspace. Channel parameters can be updated at runtime without restarting any node.
5. **Observability**: exposes an HTTP metrics API (feature: `webview`, port 3030) and a terminal TUI dashboard (feature: `tui`).

The simulator architecture is described in full in Section 5.

4.3.9 `visualization` — Browser Dashboard

A Yew/WASM application compiled to WebAssembly and served from a static HTTP server. It polls the simulator's `/node_info` endpoint on a configurable interval and renders:

- A live topology graph showing nodes and active links, with per-link channel parameters.
- Per-node traffic counters updated in real time.
- Upstream routing state for each OBU, showing the currently selected relay path toward each RSU.

The visualisation is purely read-only and stateless: all state lives in the simulator; the browser is a rendering frontend with no persistent storage.

4.3.10 `scripts_tools` — Experiment Analysis CLI

A standalone binary for processing experiment data collected from simulator runs:

`parse-band` parses bandwidth measurement logs into structured records.
`build-summary` aggregates per-node per-experiment metrics into a summary CSV.
`merge-latency` combines latency measurements from multiple experiment runs.
`ns-addrs` extracts namespace IP/MAC address assignments for correlation.
`generate-pairs` generates node-pair configuration matrices for sweep experiments.
`validate-configs` checks YAML configuration files for structural errors before running the simulator.
`autofix-configs` applies a set of automated corrections to common configuration mistakes.

4.3.11 `keygen` — Key-Generation Utility

A standalone binary for generating signing keypairs used in DH message authentication. It outputs an Ed25519 or ML-DSA-65 seed and its corresponding verifying key in hexadecimal:

```
keygen generate ed25519
keygen generate ml-dsa-65
```

The generated seed is stored in a node's YAML configuration under `signing_key_seed`, enabling stable, restartable signing identities for PKI mode (see Section 6.7.4).

4.4 Simulator Architecture in Depth

4.4.1 Network Namespace Isolation

Linux network namespaces provide the isolation primitive underlying the entire simulation. Each namespace is a separate instantiation of the kernel's network stack: it has its own loopback interface, its own routing table, its own `iptables` rule set, and its own set of sockets. Processes inside a namespace see only the interfaces assigned to that namespace.

The simulator creates one namespace per node at startup and tears them down at exit via the `netns_rs` crate's RAII guards. The namespace creation sequence for a node named `n1` is:

1. `NetNs::new("sim_ns_n1")` creates the namespace and returns a guard.
2. `nsi.run(|_| callback(args))` executes the node factory callback inside the namespace, so that all sockets and interfaces created during `callback` are bound to `n1`'s namespace.
3. TUN/TAP interfaces are created via `tun_tap::Iface`, assigned IP and MAC addresses, and brought up inside the namespace.
4. The guard is held for the duration of the simulation; dropping it would destroy the namespace and all interfaces within it.

Because each node runs in its own namespace, IP addresses can be reused across nodes (which simplifies configuration) and no routing rules in the host system are needed. The host OS sees only the namespace file descriptors, not any of the virtual interfaces inside.

4.4.2 Channel Model

Inter-node communication goes entirely through userspace channel objects rather than kernel routing. For every directed edge ($AtoB$) in the topology, the simulator maintains a `Channel` instance. The channel:

1. **Receives** raw Ethernet frames from node A's TUN write end.
2. **Filters** by destination MAC: frames not addressed to B (unicast to a different node or broadcast) are passed through; the MAC filter prevents unintended cross-talk on shared virtual media.
3. **Loss injection**: draws a uniform random float via `rand::random::<f64>()` (unseeded, non-deterministic); if the value is below the configured `loss` probability, the frame is discarded. Because the RNG is not seeded, each simulation run is statistically independent.

4. **Latency + jitter injection**: the surviving frame is enqueued with an arrival timestamp of `now + latency + U(-jitter, +jitter)`. A Tokio background task sleeps until that timestamp using `tokio_timerfd::sleep` (which uses the kernel `timerfd` interface for accurate sub-millisecond sleeps) and then writes the frame to node B's TUN read end.

Channel parameters (`latency`, `loss`, `jitter`) are stored behind an `Arc<RwLock<ChannelParams>>`. The HTTP API and TUI write to this lock; the channel task reads it before enqueuing each frame. Parameter updates therefore take effect immediately for all frames received after the update, with no node restart required.

This model accurately reproduces the first-order statistical properties of a wireless channel (mean delay, mean loss rate, jitter distribution) without implementing a physical-layer model. It is appropriate for studying routing convergence, failover behaviour, and cryptographic handshake timing, which depend on packet delivery statistics rather than PHY-level propagation effects.

4.4.3 Node Lifecycle

The simulator manages node lifecycle as follows:

1. **Creation**: `create_node_from_settings()` runs inside the namespace, creates interfaces, and returns a `Box<dyn Node>` for OBU and RSU nodes. Server nodes are started synchronously during creation and held as `Arc<Server>`. The `Node` trait exposes only `as_any()` for runtime downcasting.
2. **Starting**: OBU and RSU nodes spawn their Tokio tasks (control loop, data forwarding loop, and optional background tasks) when the simulator calls their start method on the concrete type. Server nodes begin their async tasks during creation via `block_in_place`. All tasks run on the shared Tokio runtime; no dedicated OS thread is created per node.
3. **Monitoring**: the simulator collects device and TUN statistics via `device.stats()` and `tun.stats()` on the associated interfaces. Node routing state (e.g. upstream routes) is accessed by downcasting via `as_any().downcast_ref::<Obu>()` and calling concrete methods such as `cached_upstream_route()`.
4. **Shutdown**: the simulator sends a cancellation signal to all node tasks via a `CancellationToken`. Tasks observe the cancellation and exit cleanly. Namespace guards are then dropped, destroying all virtual interfaces.

4.4.4 TUI Dashboard

The terminal TUI (feature: `tui`, built on `ratatui`) provides seven tabs accessible via number keys:

Metrics per-node counters (packets sent/received, loops detected, crypto handshakes completed, encryption errors, TAP read/write counts).

Logs a live scrolling view of `tracing` log events from all nodes, filterable by level.

Nodes a table listing each node's name, type, IP address, MAC address, namespace, and current status.

Topology an ASCII-art representation of the adjacency matrix, showing which nodes are connected and in which direction.

Channels a table of all directed links with their current `latency`, `loss`, and `jitter` values, editable in place via the TUI.

Upstreams per-OBU upstream routing state — which RSU each OBU is currently routing toward, via which relay, and the score of the cached route.

Registry the server's `obu_routes` table — the mapping from OBU virtual MAC to `(VANET MAC, RSU UDP address)` used for downstream delivery.

4.4.5 HTTP Control API

The HTTP API (feature: `webview`, port 3030) is implemented with the `warp` framework and exposes the following endpoints:

Endpoint	Method	Description
`/metrics`	`GET`	Returns a JSON object mapping node name to per-node counter values. Used by the browser dashboard and by experiment analysis scripts.
`/stats`	`GET`	Returns per-node device and TUN traffic counters (packets sent/received) aggregated across all interfaces.
`/nodes`	`GET`	Returns a list of all nodes with their name, type, IP address, and MAC address.
`/node/<name>`	`GET`	Returns detailed information for a single node identified by `<name>`.
`/channels`	`GET`	Returns the current channel parameters (`latency`, `loss`, `jitter`) for every directed link in the topology.
`/channel/<a>/<b>/`	`POST`	Updates channel parameters for the directed link from node `<a>` to node `<b>`. Body: `{"latency":"N","loss":"P","jitter":"J"}`. Takes effect immediately for all subsequent frames. Returns the updated parameters.
`/node_info`	`GET`	Returns topology metadata: node list with types and addresses, adjacency matrix, and current upstream routing state per OBU. Consumed by the browser dashboard.

Table 9 — Simulator HTTP control API endpoints

The API is stateless with respect to the browser: the simulator is the single source of truth; the browser is a read-only consumer (except for channel parameter writes via `POST /channel`).

4.5 Key Design Decisions

4.5.1 RSU Opacity

RSUs deliberately never hold session keys. All OBU payload encryption is end-to-end between the OBU and the server. Compromising an RSU exposes only routing metadata, not payload data.

This is the central security invariant of the three-tier model and directly addresses the attacker model of [1]: a compromised RSU can observe and replay routing control messages (heartbeats, HeartbeatReplies), but cannot decrypt or modify OBU payload. The RSU forwards `UpstreamForward` UDP datagrams to the server without ever possessing the session key. The server is the sole entity that can decrypt payload, and it is not exposed to the VANET medium.

4.5.2 Separation of Protocol Logic from Orchestration

A recurring problem in simulation-based research is the **fidelity gap**: the simulator contains a reimplemented approximation of the protocol, and any divergence between the approximation and the real implementation introduces potential for false results (Section 3). vigilant-parakeet eliminates this gap by design: `obu_lib`, `rsu_lib`, and `server_lib` contain the complete production node implementations, and the `simulator` crate is a thin orchestration layer that creates namespaces, instantiates these library crates, and plumbs them together with a channel model. The same library code that runs in integration tests (`cargo test`) is the code that runs in simulation and on physical hardware via the `node` binary.

The consequence is that the simulation and production codebases share a single maintenance surface. A bug fixed in `obu_lib` is fixed everywhere simultaneously.

4.5.3 Pure Route Selection

`get_route_to(Some(mac))` is a pure, read-only function. `select_and_cache_upstream()` is the sole write path for updating the cached upstream. This design was chosen deliberately to prevent hidden mutations under `RwLock` read guards — a class of bug that causes data races under Rust's type system only if the wrong lock type is used. By keeping all routing reads pure and concentrating all routing writes in a single function, the routing state is easy to reason about, test in isolation, and profile.

4.5.4 Async-First, No Threads per Node

All node I/O runs as Tokio [43] async tasks on a shared multi-threaded executor, enabling dense simulations (tens of nodes) on commodity hardware without per-node OS threads. The choice of Rust and Tokio was motivated by four properties:

Memory safety without a garbage collector. Rust's ownership and borrow checker eliminates entire classes of memory-safety bugs (buffer overflows, use-after-free, data

races) at compile time, without introducing the latency jitter of garbage collection. For a timing-sensitive routing evaluation, GC pauses would be confounding variables.

Zero-cost abstractions. Async Rust compiles `async fn` coroutines to state machines with no dynamic allocation per `.await`. The per-task overhead of a Tokio task is a single heap allocation for the stack frame — typically under 256 bytes — making it practical to run dozens of concurrent I/O loops per node without resource pressure.

Natural latency injection. `tokio::time::sleep(duration)` correctly models a link delay without blocking any OS thread. Tokio's time facilities also support **mocked time** (via `tokio::time::pause()` and `advance()`), which the test infrastructure exploits to drive deterministic integration tests at nanosecond precision without wall-clock waits.

Ecosystem. The `tokio`, `bytes`, `mac_address`, `indexmap`, `rand`, and `warp` crates in the Rust ecosystem provide the building blocks for all major subsystems without requiring any unsafe code in the node libraries.

4.5.5 Feature-Gated Instrumentation

The `stats` feature compiles in atomic counters (`loop_detected_count`, `encrypted_frames_sent`, etc.) at zero cost when disabled. The `webview` and `tui` features are similarly additive. This design pattern — compile-time feature flags rather than runtime configuration — ensures that production builds carry no instrumentation overhead, while experiment builds have full observability.

The `test_helpers` feature unlocks in-process hub and TUN shim implementations used by the integration test suite. It is never compiled into production builds.

4.5.6 Test Infrastructure Without Root

The `test_helpers::Hub` and `TokioTun` shim allow full integration tests of multi-hop routing, encryption, and failover without creating kernel interfaces or requiring elevated privileges. The core insight is that the routing and encryption logic does not depend on the mechanism by which frames are sent and received — only on the interface (the `Device` and `Tun` traits). By providing trait implementations backed by in-memory channels, the test suite can exercise the full control plane deterministically and without OS-level side effects.

This design makes the test suite runnable in CI without `sudo`, which is critical for automated testing in standard container environments (GitHub Actions, GitLab CI). The simulator itself — which creates real namespaces — is excluded from CI coverage precisely because it requires privileges, but the library logic it orchestrates is fully covered.

4.5.7 Why Userspace Channel Emulation over `tc-netem`

The Linux kernel's `tc-netem` module (Netlink-configurable network emulator) provides packet delay, loss, duplication, and corruption in the kernel fast path. It would be a natural substrate for link emulation. vigilant-parakeet instead implements a userspace `Channel` object for three reasons:

First, `tc-netem` rules are attached to a **kernel network interface**, which means they require privileged `ip link` / `tc` invocations to configure. The simulator would need additional privilege grants or a privileged helper process for every channel parameter update. The userspace approach updates parameters via an `Arc<RwLock<ChannelParams>>` with no system call overhead.

Second, `tc-netem` distributes jitter using a pre-compiled kernel distribution table (loaded via `tc dist`), making runtime distribution changes expensive. The userspace model makes distribution parameters a simple struct field change.

Third, and most importantly, the userspace channel cooperates with Tokio's mocked time. `tc-netem` uses the real kernel clock for delay injection; a test using `tokio::time::pause()` to fast-forward time would not advance `tc-netem` delays. The userspace `tokio_timerfd::sleep` calls do advance under mocked Tokio time, enabling sub-millisecond precision in integration tests.

5. Implementation

5.1 Wire Protocol

5.1.1 Message Types

All VANET inter-node communication uses fixed-format binary messages in `node_lib::messages`:

`Heartbeat` Emitted periodically by RSUs. Carries the RSU's uptime at emission time (`duration`), a monotonically increasing sequence number (`id`), a hop counter incremented by each relay, and the RSU MAC address (`source`).

`HeartbeatReply` Sent by an OBU (or relay) in response to a received `Heartbeat`. Copies `duration`, `id`, `hops`, and `source` from the original Heartbeat, and appends the replying node's MAC as `sender`. The RSU receiving the reply computes round-trip latency as `now_uptime - duration`.

`Data::{Upstream, Downstream}` Payload wrappers carrying a (potentially encrypted) byte buffer with source and destination MAC addresses.

`KeyExchangeInit` / `KeyExchangeReply` Variable-length DH/KEM handshake messages. Size depends on the configured algorithm: 45 bytes unsigned for X25519, 1197 bytes for ML-KEM-768 Init, or 1101 bytes for ML-KEM-768 Reply. An optional signed extension appends the signing public key and signature, adding 101 bytes for Ed25519 or up to 5266 bytes for ML-DSA-65 (see Section 6 for full wire format detail).

`Message` Outer container with a 1-byte type discriminant followed by the serialised inner message.

All types implement `TryFrom<&[u8]>` for zero-copy deserialisation and `Into<Vec<u8>>` for serialisation.

5.1.2 Heartbeat Wire Layout

duration (16 B)	id (4 B)	hops (4 B)	source (6 B)

Table 10 — Heartbeat wire format (30 bytes, big-endian)

All fields are big-endian. `duration` is the sender's uptime in milliseconds at emission time (16-byte BE integer), used by receivers to timestamp the arrival and compute round-trip latency. `id` is the monotonically increasing sequence number. `hops` is incremented by each

relay on the forward path. `source` is the RSU's MAC address, used as the primary routing table key.

5.1.3 Cloud Protocol Wire Layout

RSU–Server communication uses a separate binary protocol over UDP (`server_lib::cloud_protocol`):

`MAGIC (2 B)` `0xAB 0xCD`	`TYPE (1 B)`	payload (fields vary by type — see Table 12)

Table 11 — Cloud protocol message header

Type byte	Message	Fields after MAGIC+TYPE
`0x01`	`RegistrationMessage`	`RSU_MAC (6B), OBU_COUNT (2B), OBU_MAC×N`
`0x02`	`UpstreamForward`	`RSU_MAC (6B), OBU_SRC_MAC (6B), PAYLOAD`
`0x03`	`DownstreamForward`	`OBU_DST_MAC (6B), ORIGIN_MAC (6B), PAYLOAD`
`0x04`	`KeyExchangeForward`	`RSU_MAC (6B), CONTROL_PAYLOAD`
`0x05`	`KeyExchangeResponse`	`OBU_DST_MAC (6B), RESPONSE_PAYLOAD`

Table 12 — Cloud protocol message types

5.2 Routing Protocol

5.2.1 Heartbeat Emission (RSU)

An RSU emits a `Heartbeat` every `hello_periodicity` milliseconds to the broadcast MAC address `ff:ff:ff:ff:ff:ff`. The sequence number increments with each emission. The RSU is stateless with respect to upstream routing.

5.2.2 Heartbeat Reception and Route Building (OBU)

When an OBU receives a `Heartbeat`:

1. It records an entry in its routing table keyed by `(rsu_mac, seq_id)`, storing `pkt.from`, hop count, and arrival timestamp. The table is an `IndexMap` bounded by `hello_history`, so stale entries are evicted automatically.
2. If the OBU has downstream nodes, it increments the hop counter and rebroadcasts.
3. It emits a `HeartbeatReply` toward the RSU.

5.2.3 Route Selection Metric

`get_route_to(Some(target))` in `obu_lib::control::routing` proceeds as follows.

When latency measurements are available, each candidate next-hop m is scored by:

$$s_m = \mu_m^{\min} + \overline{\mu}_m$$

where $\mu_m^{\min}$ is the minimum observed round-trip latency to m (in microseconds) and $\overline{\mu}_m$ is the mean latency across all recorded observations to m, both derived from the `PerHopInfo` entries stored in the `RoutingTable` (`HashMap<MacAddress, IndexMap<u32, PerHopInfo>>`). The candidate with the lowest score is selected. Using both the minimum and the mean penalises candidates that are occasionally fast but highly variable, while still rewarding consistently low-latency paths. Round-trip times are computed at the RSU as `Instant::now().duration_since(boot) - duration`, where `duration` is the RSU uptime embedded in the original Heartbeat, and propagated back to the OBU in its reply. Ties in the integer microsecond score are broken deterministically by MAC address lexicographic order, ensuring a stable selection across equivalent candidates.

When no latency measurements have been recorded yet (e.g.
at startup before any HeartbeatReply timing has been collected), the algorithm falls back to preferring the candidate with the fewest advertised hops.

The **cached upstream** is retained via a 10% hysteresis threshold: the algorithm only switches to a new candidate if its score is at least 10% lower than the cached candidate's score, or if it has strictly fewer hops. This prevents frequent route flipping caused by transient latency variance without preventing recovery from genuinely better paths.

Implementation details:

- `get_route_to(Some(target))` is pure and computes scores from read-only heartbeat state.
- `select_and_cache_upstream()` performs the single write to update the cached upstream and stores an N-best ordered list (default N=3) for fast failover.
- Failover promotes the head of the N-best list if the active upstream fails or exhibits timeouts. Each candidate includes timestamped measurements so stale entries age out.

5.2.4 N-Best Candidate Caching

`select_and_cache_upstream(mac)` scores every known next-hop candidate using the same `s_m = mu_m^min + overline(mu)_m` metric as the primary selection, then stores the top `cached_candidates` (default: 3) in a ranked `Vec<CachedCandidate>`. Each `CachedCandidate` carries the next-hop MAC, the score used for ordering, and the measurement timestamp that was current at selection time.

`failover_cached_upstream()` is called when the active upstream fails (e.g.
a send error or a timeout). It promotes the head of the candidate list to primary and removes it from the list, all under a single write-lock acquisition and with no re-scan of the routing table. This makes failover O(1) in the common case.

The candidate list is implicitly age-bounded by `hello_history`: the routing table is itself an `IndexMap` bounded to `hello_history` entries per RSU–sender pair. When entries are evicted by new heartbeats, their associated latency samples disappear; a candidate whose samples have all aged out will score worse than a fresh candidate at the next

`select_and_cache_upstream` call. The list is rebuilt in full on every primary-route update triggered by the 10% hysteresis threshold check.

5.2.5 Loop Prevention

HeartbeatReply forwarding is controlled by a `ForwardAction` enum computed at each relay OBU before any I/O takes place:

`Bail` Drop the reply immediately. Triggered when no upstream is known yet, or when the cached upstream is the node that sent the reply (immediate-bounce guard: `pkt.from == next_upstream`). Sending the reply back to its origin would create a two-node oscillation.

`SkipForward` The selected upstream matches the message's `sender` field (`next_upstream == message.sender()`), which would create a forwarding loop. The reply is dropped; the relay then attempts `failover_cached_upstream()` to promote the next N-best candidate.

`Forward` Normal case. The reply is forwarded to `next_upstream`, which differs from both `pkt.from` and `message.sender()`.

`ForwardCached` Used when the upstream was not yet cached at dispatch time but was resolved from the N-best list during the forward attempt. Semantically equivalent to `Forward`; the distinction allows the metrics layer to count cache-miss forwards separately.

The `ForwardAction` calculation is pure (reads routing state under a read lock) and is performed before acquiring any write lock, keeping the hot-path lock scope small.

5.2.6 Replay Detection

HeartbeatReply messages carry the original Heartbeat sequence number back to the RSU so that the RSU can compute round-trip latency and update routing metrics. Without replay protection, an attacker that captures a legitimate HeartbeatReply can re-inject it later to make the RSU believe a routing path is fresher or lower-latency than it really is, maintaining stale routing entries and manipulating route selection.

vigilant-parakeet implements a **per-sender sliding receive window** on HeartbeatReply sequence numbers at the RSU (`rsu_lib::control::routing::ReplayWindow`), following the same design as IPsec AH [36]:

- Each `(RSU, sender MAC)` pair has its own independent `ReplayWindow`.
- The window state is a `(last_seq: u32, window: u64)` pair. `last_seq` is the highest accepted sequence number; `window` is a 64-bit bitmask where bit i is set when sequence number `last_seq - i` has been accepted.
- The window width is 64 sequence numbers. A reply with sequence number `seq` is accepted if and only if `seq > last_seq - 64` and `seq` has not been accepted before.
- On acceptance of a new highest sequence number, the bitmask is left-shifted by the advance and bit 0 is set for the new entry.
- Replies with `seq <= last_seq - 64` (outside the window) are silently dropped.
- Replies with a `seq` already recorded in the bitmask are silently dropped as duplicates.

A subtle additional guard prevents a **window-poisoning attack**: before `check_and_update` is called, the RSU verifies that `seq` appears in its own sent-heartbeat history. Without this guard, an attacker could forge a reply with `seq = u32::MAX`, advance `last_seq` to `u32::MAX`, and cause all subsequent legitimate replies from that sender to fall outside the window, effectively denying routing updates from a benign OBU. The sequence wraparound case (`u32` overflow) is handled by clearing all window state when the RSU detects that its own sequence counter has wrapped.

The `ReplayWindow` is unit-tested in `rsu_lib/src/control/routing.rs` via `replay_window_tests` (same-sequence rejection, outside-window rejection) and integration-tested via `replay_integration_tests` (replayed reply does not insert duplicate route, forged large ID does not poison window).

5.3 End-to-End Data Path

The full data path for an OBU sending a packet to an application server illustrates how the three tiers interact (see figure).

`Data::Upstream` and `Data::Downstream` are the VANET payload wrappers used on the tier-1 medium. They are wrapped inside the outer `Message` container (1-byte type discriminant), and their own layout depends on direction:

Variant	Header fields	Followed by
`Upstream` (`0x00`)	`origin (6 B)`	AEAD-encrypted payload (nonce + ciphertext + auth tag)
`Downstream` (`0x01`)	`origin (6 B) + dst (6 B)`	AEAD-encrypted payload

Table 13 — `Data` wire format inside the `Message` container; no length prefix — the payload occupies all remaining bytes

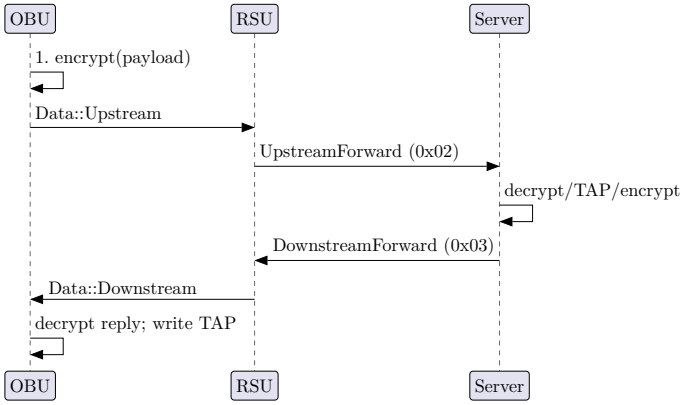

Figure 3 — End-to-end data path across all three tiers

5.4 Simulator Orchestration

5.4.1 Network Namespace Setup

For each node the simulator creates an isolated network namespace using `netns_rs::NetNs::new("sim_ns_<name>")` and executes the node factory callback inside it via `nsi.run(|_| callback(...))`, giving each node its own independent network stack.

Per-link channel conditions (latency, loss, jitter) are simulated entirely in userspace by `simulator::channel::Channel` — no kernel `tc netem` rules are involved. Each directed link is backed by a `Channel` instance that:

1. **Filters** incoming frames by MAC address (unicast to this node or broadcast).
2. **Drops** packets probabilistically when `rand::random::<f64>() < loss`.
3. **Enqueues** surviving packets with their arrival timestamp and forwards them to the destination TUN interface after sleeping for `latency ± jitter` using `tokio_timerfd::sleep`, implemented as a background Tokio task.

Channel parameters can be updated at runtime (taking effect immediately for new packets) via the HTTP API or TUI, without restarting any node.

5.4.2 `node_factory` Module

`simulator::node_factory::create_node_from_settings()` creates the correct set of interfaces and node instance inside the namespace context:

- **OBU**: `vanet` TAP + `virtual` TAP → `obu_lib::create_with_vdev(args, tun, device, name)`
- **RSU**: `vanet` TAP + `cloud` TAP (UDP socket bound here) → `rsu_lib::create_with_vdev(args, device, name)`
- **Server**: `virtual` TAP + `cloud` TAP (UDP socket) → `Server::new(...).with_tun(tun)`, `server.start()` called immediately via `block_in_place`

5.4.3 HTTP Control API (feature: `webview`)

Endpoint	Method	Description
`/metrics`	`GET`	JSON per-node counter values
`/stats`	`GET`	Per-node device and TUN traffic counters
`/nodes`	`GET`	Node list with name, type, IP, and MAC
`/node/<name>`	`GET`	Detailed info for a single node
`/channels`	`GET`	Current channel parameters for all directed links
`/channel/<a>/<b>/`	`POST`	`{"latency":"N","loss":"P","jitter":"J"}` — update per-link channel parameters at runtime
`/node_info`	`GET`	Topology and upstream routing state for the browser dashboard

Table 14 — Simulator HTTP control API endpoints

5.5 TCP Admin Console

Each node type exposes a TCP admin interface bound to `127.0.0.1:<admin_port>` inside its network namespace (default port: 9000). The interface is a line-oriented text protocol, making it reachable via standard tools without any additional client software:

```
ip netns exec <node_ns> nc 127.0.0.1 9000
ip netns exec <node_ns> telnet 127.0.0.1 9000
```

The `admin_port` key in each node's YAML configuration selects the listening port. Multiple concurrent connections are accepted; each session is independent. All responses use CRLF line endings for terminal compatibility.

Because the bind address is inside the network namespace, the admin interface is not reachable from outside the simulation host without explicit namespace entry — it serves as a local operations channel rather than a network-exposed management API.

5.5.1 OBU Admin Commands

Command	Description
`info`	Node identity (name, MAC), current upstream next-hop, and DH session status.
`session`	Active DH session: `key_id`, age since establishment, and whether a re-key is in progress.
`routes`	Cached upstream candidates in priority order (primary + fallback entries).
`rekey`	Immediately clears the current DH session and triggers a new key exchange. Useful for manual session rotation or debugging.
`quit`	Close the admin connection.

Table 15 — OBU admin console commands

5.5.2 RSU Admin Commands

Command	Description
`info`	Node identity (name, MAC), number of known OBU clients, and VANET route count.
`clients`	Table of known OBU clients: OBU VANET MAC and the next-hop MAC through which the RSU reached each OBU.
`routes`	VANET next-hop routing table: next-hop MAC, hop count, and observed latency.
`quit`	Close the admin connection.

Table 16 — RSU admin console commands

5.5.3 Server Admin Commands

Command	Description
`sessions`	Lists all active DH sessions: OBU VANET MAC, `key_id`, and session age.
`revoke <mac>`	Terminates the DH session for the specified OBU. Clears the server-side key and sends a signed `SessionTerminated` message via the RSU to the OBU, which immediately initiates a fresh key exchange (see Section 6.8).
`routes`	OBU routing table: virtual TAP MAC, VANET MAC, and the RSU UDP address used for downstream delivery.
`registry`	RSU registration table: RSU MAC mapped to its currently associated OBU MACs.
`allowlist`	PKI signing allowlist: OBU VANET MACs and their pre-registered verifying keys (empty if PKI mode is not configured).
`quit`	Close the admin connection.

Table 17 — Server admin console commands

5.6 Visualisation Dashboard

The browser-based visualisation dashboard (`visualization/`) is a Yew/WebAssembly application compiled to WASM and served alongside the simulator HTTP API. It provides a live read-only view of simulation state without modifying any node behaviour, making it suitable for demonstration and monitoring without experimental side effects.

5.6.1 Technology Stack

The frontend is written in Rust using the **Yew** framework — a component-based web UI library analogous to React, but compiled from Rust to WebAssembly via `wasm-bindgen`. Building and serving the dashboard requires `trunk`, a Rust WASM bundler:

```
rustup target add wasm32-unknown-unknown
cargo install trunk

cd visualization
trunk build --release   # produces dist/ with index.html + wasm bundle
trunk serve             # dev server with live reload
```

The Yew component model maps directly onto the simulator's data model: `NodeState`, `ChannelState`, and `UpstreamState` structs are shared with the HTTP API layer and serialised as JSON. The dashboard fetches the `/node_info` endpoint on a configurable polling interval (default 1 second) and triggers a Yew re-render on each update.

5.6.2 Dashboard Components

The dashboard renders three primary views:

Topology graph: an SVG force-directed graph showing nodes (OBUs in one colour, RSUs in another, server in a third) connected by directed edges representing the current topology. Each edge label shows the configured channel latency and loss for that directed link. Clicking a node opens an info panel showing its IP address, MAC address, current upstream route, and DH session status.

Traffic counters: a table updated on each poll showing per-node packet counters (`frames_sent`, `frames_received`, `encrypted_frames`, `tap_writes`) from the `/metrics` endpoint. Counters are displayed as absolute values and as per-second deltas computed from successive polls, giving an at-a-glance throughput view.

Upstream routing state: a per-OBU panel showing the currently selected upstream relay MAC and the N-best candidate list. When a failover occurs (primary candidate promoted to head, new head selected), the UI highlights the changed entry for one rendering cycle. This makes the failover mechanism observable in real time during manual experiments.

5.6.3 Architectural Separation

The dashboard is architecturally separate from the simulator: it communicates exclusively via the HTTP API and makes no assumptions about the simulator's internal implementation. The only shared artefact is the JSON schema of the API responses, which is defined as Rust structs in `simulator/src/webview.rs` and serialised via `serde_json`. This separation means the dashboard can be updated or replaced without modifying the simulator, and the simulator can run without a browser client.

The WebAssembly target is fully separate from the simulator's compilation: `cargo build --workspace` does not include the `visualization` crate (it requires the `wasm32-unknown-unknown` target and `trunk`). CI builds the WASM artifact in a separate step from the native simulator.

5.7 Simulator Configuration

The simulator is configured via two layers of YAML files: a top-level **topology file** describing the simulation scenario, and per-node **node configuration files** specifying each node's parameters.

5.7.1 Topology File Format

The topology file controls scenario structure:

```yaml
# simulator.yaml
nodes:
  rsu1:
    config_path: rsu1.yaml
  obu1:
    config_path: obu1.yaml
```

```yaml
  obu2:
    config_path: obu2.yaml
  server:
    config_path: server.yaml

topology:
  rsu1:
    obu1:
      latency: 10    # ms
      loss: 0.0      # probability 0.0–1.0
      jitter: 2      # ms
    obu2:
      latency: 50
      loss: 0.05
      jitter: 5
  obu1:
    rsu1:
      latency: 10
      loss: 0.0
      jitter: 2
    obu2:
      latency: 5
      loss: 0.0
      jitter: 1
```

Each entry under `topology` is a directed edge with `latency` (ms), `loss` (probability 0.0–1.0), and `jitter` (ms half-range). Edges are unidirectional: `rsu1 → obu1` and `obu1 → rsu1` are configured separately, allowing asymmetric link conditions.

The YAML values are parsed at startup and used to initialise `Channel` instances for each directed edge. Channel parameters can be updated at runtime (see Section 5.2) via the HTTP API without changing or reloading the file.

5.7.2 Node Configuration File Format

Each node has an individual YAML file specifying its type and operational parameters:

```yaml
# obu1.yaml
node_type: Obu
ip: 10.0.0.2            # VANET IP address
admin_port: 9001        # TCP admin console port

# Routing parameters
hello_history: 10       # heartbeat entries per peer
cached_candidates: 3    # N-best upstream candidates

# DH key exchange parameters
dh_rekey_interval_ms: 43200000   # 12 hours
dh_key_lifetime_ms:   86400000   # 24 hours
dh_reply_timeout_ms:  5000       # 5 seconds
```

```yaml
# Crypto configuration
cipher: aes-256-gcm     # aes-256-gcm | aes-128-gcm | chacha20-poly1305
dh_group: x25519        # x25519 | ml-kem-768
mtu: 1400               # max TAP frame size

# Optional: DH message authentication
# enable_dh_signatures: true
# signing_algorithm: ed25519       # ed25519 | ml-dsa-65
# signing_key_seed: "<64-hex>"
# server_signing_pubkey: "<hex>"
```

```yaml
# rsu1.yaml
node_type: Rsu
ip: 10.0.0.1
hello_periodicity: 500  # ms between Heartbeats
hello_history: 10
admin_port: 9000
```

```yaml
# server.yaml
node_type: Server
ip: 172.0.0.1           # cloud-tier IP
admin_port: 9100
# Optional: signing allowlist for PKI mode
# dh_signing_allowlist:
#   "AA:BB:CC:DD:EE:FF": "<hex verifying key>"
```

The configuration system allows a single experiment directory to contain multiple topology files targeting different channel conditions or node counts, with all nodes sharing the same per-node YAML. This supports systematic parameter sweeps: the `scripts_tools::generate-pairs` command generates topology matrices across a range of latency and loss values for automated batch experiments.

5.8 Test Infrastructure

5.8.1 The `Hub` Helper

`node_lib::test_helpers::hub::Hub` is an in-process programmable switch backed by raw UNIX socket pairs (one pair per attached node). Its main characteristics are:

- **Frame delivery**: the Hub's poll loop calls `libc::recv(fd, MSG_DONTWAIT)` on each node's socket in a tight loop with 100 µs sleeps, forwarding each frame to all other attached nodes after an optional per-hop delay via `tokio::time::sleep`. Because this uses Tokio's time primitives, the delay respects mocked time (via `tokio::time::pause()` / `advance()`), making deterministic latency tests possible without real wall-clock waits.

- **The `HubCheck` trait**: tests implement `HubCheck` (with a single `on_packet(from_idx, data)` method) and attach the checker as a watch hook. The Hub calls the hook for every forwarded frame, allowing precise per-packet assertions without any changes to production node code.
- **Loss injection**: per-link loss is applied with a seeded `rand::rngs::SmallRng`, giving deterministic packet-drop sequences across test runs.
- **`UpstreamExpectation`**: a helper that wraps a `tokio::sync::oneshot::Receiver` and resolves when a matching frame has been observed, enabling `await`-based test assertions.

5.8.2 TUN Shim

`common::Tun::new_shim()` returns a `(TokioTun, FrameChannel)` pair backed by Tokio channels. Tests inject frames into nodes and assert on outputs with zero OS-level side effects and no root privileges required.

5.8.3 Integration Test Coverage

`node_lib/tests/` contains nine integration test files; each targets a specific protocol property:

`integration_flow` Verifies that the base `handle_messages` dispatch table correctly routes incoming frames to the TUN device or VANET device according to message type.

`integration_topology` Creates a minimal RSU + OBU topology and asserts that the OBU discovers a valid upstream route to the RSU after receiving its first Heartbeat.

`integration_two_hop` Places two OBUs in a chain (OBU_1 — OBU_2 — RSU) and confirms that OBU_2 prefers the two-hop path through OBU_1 when the direct link carries higher simulated latency.

`integration_encryption` Uses mocked time to drive the full DH key exchange between an OBU and the server, then asserts that TAP frames sent by the OBU arrive at the server decrypted. Verifies that an eavesdropping relay OBU sees only ciphertext.

`integration_broadcast_encryption` Confirms that broadcast traffic sent by the server remains opaque to intermediate relay OBUs that do not hold the session key.

`integration_failover_send_error` Injects a send error on the primary upstream link and asserts that the OBU immediately promotes the next N-best candidate, restoring end-to-end frame delivery without waiting for a new Heartbeat cycle.

`integration_loop_repro` Reproduces a historical loop condition in a diamond topology and asserts that the `ForwardAction` guards prevent any HeartbeatReply from being forwarded indefinitely.

`integration_latency_measurement_mocked_time` Uses `tokio::time::advance` to simulate precise round-trip intervals and asserts that the RSU computes the correct RTT from the `duration` field of the HeartbeatReply.

`routing_integration` End-to-end routing-table convergence test: verifies that after several Heartbeat cycles the routing table contains the expected next-hop entries and that the hysteresis threshold prevents spurious route changes under small latency perturbations.

`server_lib/tests/integration_encryption.rs` covers the server-side DH key exchange and decryption path end-to-end, including ML-KEM-768 and X25519 variants.

6. Security Architecture

A key motivation for this project is the study of Layer-3 security in vehicular networks [1]. vigilant-parakeet therefore implements a full end-to-end security layer enabling OBUs to establish encrypted sessions with a backend server without trusting intermediate RSUs or relay OBUs.

6.1 Threat Model

The security model assumes the following adversary capabilities:

- **Untrusted relay nodes**: intermediate OBUs and even RSUs may be compromised. They must not be able to read or tamper with payload data exchanged between a legitimate OBU and the server.
- **Passive eavesdroppers**: an attacker can capture all frames on any link in the network.
- **Active man-in-the-middle**: an adversary positioned between the OBU and the server can intercept and modify in-flight key exchange messages. In the absence of authentication, this allows the attacker to substitute its own public key and establish independent DH sessions with each endpoint, recovering plaintext as a transparent relay.
- **Routing manipulation**: as analysed in [1], an attacker controlling intermediate nodes can inject or replay heartbeat control messages to poison OBU routing tables, steering traffic through attacker-controlled paths without breaking the encryption layer.
- **Quantum adversary**: a computationally unbounded adversary equipped with a large-scale quantum computer can break X25519 and Ed25519 using Shor's algorithm [32], recovering session keys from stored ciphertext or forging handshake signatures. The **harvest now, decrypt later** (HNDL) attack — capturing traffic today for quantum-assisted decryption in the future — is a present concern given the long deployment lifetimes of vehicular infrastructure. ML-KEM-768 and ML-DSA-65 are provided as drop-in replacements for X25519 and Ed25519 respectively, restoring security under the Module Learning With Errors (MLWE) hardness assumption (see Section 2.4.7).

The design consequence is **end-to-end encryption at the OBU–server boundary**: RSUs and relay OBUs forward encrypted payloads opaquely and never hold the session key. The MitM threat is addressed by an optional Ed25519 authentication layer on the key

exchange messages (see Section 6.7). Control-plane heartbeat authentication is identified as future work (Section 8).

6.2 Configurable Cipher Suite

All cryptographic parameters are encapsulated in `CryptoConfig` (defined in `node_lib::crypto`):

```
pub struct CryptoConfig {
    pub cipher:            SymmetricCipher,   // default: AES-256-GCM
    pub kdf:               KdfAlgorithm,      // default: HKDF-SHA256
    pub dh_group:          DhGroup,           // default: X25519
    pub signing_algorithm: SigningAlgorithm,  // default: Ed25519
}
```

Each axis is independently configurable via node YAML:

Parameter	Options	Notes

`cipher`	`aes-256-gcm` (default)
	`aes-128-gcm`
	`chacha20-poly1305`
		All three are AEAD ciphers providing both confidentiality and integrity. ChaCha20-Poly1305 is preferred on hardware without AES-NI.

`kdf`	`hkdf-sha256` (default)
	`hkdf-sha384`
	`hkdf-sha512`
		HKDF [29] extracts and expands the raw DH shared secret into a uniformly random key of the required length.

`dh_group`	`x25519` (default) `ml-kem-768`	X25519 (Curve25519 [28]) provides 128-bit classical security. ML-KEM-768 (NIST FIPS 203 [31]) is the quantum-resistant alternative, providing security against attacks by quantum computers.
`signing_algorithm`	`ed25519` (default) `ml-dsa-65`	Ed25519 [30] is the classical signing algorithm. ML-DSA-65 (NIST FIPS 204 [33]) is the quantum-resistant alternative. The signing algorithm applies to both key exchange authentication and session revocation.

Table 18 — Configurable cipher suite parameters

6.3 Key Exchange Protocol

Key exchange follows a two-message Diffie-Hellman handshake carried over the existing control-plane message infrastructure.

6.3.1 Wire Format

Two control message types are defined in `node_lib::messages::control::key_exchange`. Both `KeyExchangeInit` and `KeyExchangeReply` share the same variable-length layout; the distinguishing information is the message type discriminant in the outer `Message` container. An `algo_id` byte at the start of each message identifies the key-exchange algorithm in use, enabling receivers to parse key material of the correct length.

Unsigned base format:

Field	Size	Description
`algo_id`	1 B	Key-exchange algorithm: `0x01` = X25519, `0x02` = ML-KEM-768.
`key_id`	4 B	Monotonically increasing exchange identifier, used as HKDF salt to bind the derived key to this round.
`key_material_len`	2 B BE	Length of the following key material field.
`key_material`	var	For X25519: 32-byte ephemeral public key. For ML-KEM-768 Init: 1184-byte encapsulation key. For ML-KEM-768 Reply: 1088-byte ciphertext.
`sender`	6 B	MAC address of the originating node.

Table 19 — Unsigned `KeyExchangeInit` / `KeyExchangeReply` base format (variable length)

Total unsigned size: 45 bytes for X25519 (1+4+2+32+6), 1197 bytes for ML-KEM-768 Init (1+4+2+1184+6), and 1101 bytes for ML-KEM-768 Reply (1+4+2+1088+6).

Signed extension: when authentication is enabled (see Section 6.7), a variable-length extension is appended after the base payload:

Field	Size	Description
`sig_algo_id`	1 B	Signing algorithm: `0x01` = Ed25519, `0x02` = ML-DSA-65.
`spk_len`	2 B BE	Length of the signing public key.
`signing_pubkey`	var	Ed25519: 32 bytes. ML-DSA-65: 1952 bytes.
`sig_len`	2 B BE	Length of the signature.
`signature`	var	Signature over the base payload. Ed25519: 64 bytes. ML-DSA-65: 3309 bytes.

Table 20 — Signed extension appended to `KeyExchangeInit` / `KeyExchangeReply`

The `TryFrom<&[u8]>` deserialiser reads `algo_id` and `key_material_len` first, then parses the key material of the declared length, followed by the 6-byte sender. If bytes remain, the signed extension is parsed and the `sig_algo_id` is validated against the known Ed25519 and ML-DSA-65 sizes; any mismatch is rejected. The cloud-protocol `KeyExchangeForward` (type `0x04`) and `KeyExchangeResponse` (type `0x05`) messages carry the raw payload opaquely, so they are algorithm-agnostic.

6.3.2 Handshake Flow

The handshake is a two-message exchange, but the mechanics differ by algorithm:

6.3.2.1 X25519 (Classical ECDH)

1. The OBU generates an ephemeral X25519 keypair, assigns the next `key_id`, stores a `PendingExchange`, and sends `KeyExchangeInit` carrying its 32-byte public key toward the server.
2. The RSU forwards the message opaquely as a `KeyExchangeForward` UDP datagram (type `0x04`) to the server.
3. The server generates its own ephemeral X25519 keypair, computes `DH(server_secret, obu_public)`, derives the session key via HKDF, stores it, and returns its 32-byte public key in `KeyExchangeReply`.
4. The OBU receives the reply, computes `DH(obu_secret, server_public)`, derives the same session key via HKDF, and moves the entry to `established`.

Both sides arrive at the same shared secret without transmitting any private key material.

6.3.2.2 ML-KEM-768 (Quantum-Resistant KEM, NIST FIPS 203)

ML-KEM-768 is a Key Encapsulation Mechanism (KEM): rather than two parties performing a joint Diffie-Hellman computation, one party (the server) **encapsulates** a fresh random secret under the other's public key (the encapsulation key), and the holder of the corresponding decapsulation key recovers the secret.

Its security rests on the **Module Learning With Errors** (MLWE) problem (see Section 2.4.7): an attacker who observes the encapsulation key and the ciphertext must solve an instance of MLWE to recover the shared secret, a problem for which no efficient classical or quantum algorithm is known. This gives ML-KEM-768 IND-CCA2 security at NIST Security Level 3 — comparable to breaking AES-192 — under the MLWE assumption.

1. The OBU generates an ML-KEM-768 keypair, retains the 64-byte decapsulation-key seed, and sends its 1184-byte **encapsulation key** in `KeyExchangeInit`.
2. The server calls `kem_768_encapsulate(encap_key)`, which produces a 1088-byte **ciphertext** and a 32-byte shared secret. The server derives the session key from the shared secret via HKDF and sends the ciphertext in `KeyExchangeReply`.
3. The OBU reconstructs its decapsulation key from the stored seed and calls `kem_768_decapsulate(seed, ciphertext)` to recover the same 32-byte shared secret, then derives the session key via HKDF.

The shared secret is chosen by the server and sent to the OBU encrypted under its encapsulation key. A passive or active attacker without the decapsulation key cannot recover the secret even with a quantum computer, because breaking the ciphertext requires solving MLWE. Note the role asymmetry compared to X25519: the OBU cannot verify that the encapsulated secret was chosen freshly (rather than replayed) without the authenticated handshake extension (Section 6.7), making authentication especially important in the ML-KEM mode.

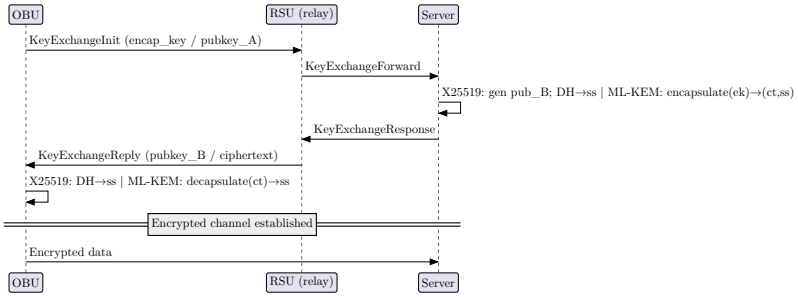

Figure 4 — Key exchange handshake (unsigned): X25519 ECDH or ML-KEM-768 KEM, and subsequent encrypted data flow

6.4 Key Derivation

The raw shared secret — 32 bytes from either X25519 or ML-KEM-768 — is not used directly as a symmetric key. Instead, it is passed through HKDF [29]:

```
HKDF-Expand(
    PRK  = HKDF-Extract(salt, shared_secret),
    info = b"vigilant-parakeet-dh",
    L    = cipher.key_len()    // 16 (AES-128) or 32 (AES-256, ChaCha20)
)
```

The HKDF salt is constructed as: `b"vigilant-parakeet-salt-" ‖ key_id.to_be_bytes()`, which ensures that re-keying (incrementing `key_id`) produces a cryptographically independent key even if the same keypair were reused. Using HKDF as a post-processing step is particularly important for ML-KEM-768: the KEM shared secret is pseudorandom by construction, but routing it through HKDF ensures uniform key material regardless of any subtle biases in the KEM output and allows the same derivation code path for both algorithms.

6.5 Symmetric Encryption

Payload encryption uses AEAD (Authenticated Encryption with Associated Data). The wire format for every encrypted buffer is:

`nonce (12 B)`	`ciphertext (variable)`	`auth tag (16 B)`

Table 21 — AEAD encrypted buffer format; total overhead: 28 bytes

The 12-byte nonce is generated freshly from the OS CSPRNG for every encryption, ensuring that nonce reuse is eliminated even under high traffic rates. The 16-byte GCM/Poly1305 authentication tag provides integrity and authenticity; decryption fails immediately if the tag does not verify, preventing tampering by any relay node.

The encryption path in `obu_lib::control` is:

```
// Upstream: OBU → server
let encrypted = encrypt_with_config(cipher, plaintext, &dh_key)?;

// Downstream: server → OBU
let plaintext = decrypt_with_config(cipher, buf.data(), &dh_key)?;
```

If no established key exists for the server (handshake not yet complete), encryption is skipped and the frame is held or dropped depending on configuration.

6.6 DH Key Store Lifecycle

The `DhKeyStore` in `obu_lib::control::dh_key_store` manages key state as a simple state machine (Figure 5):

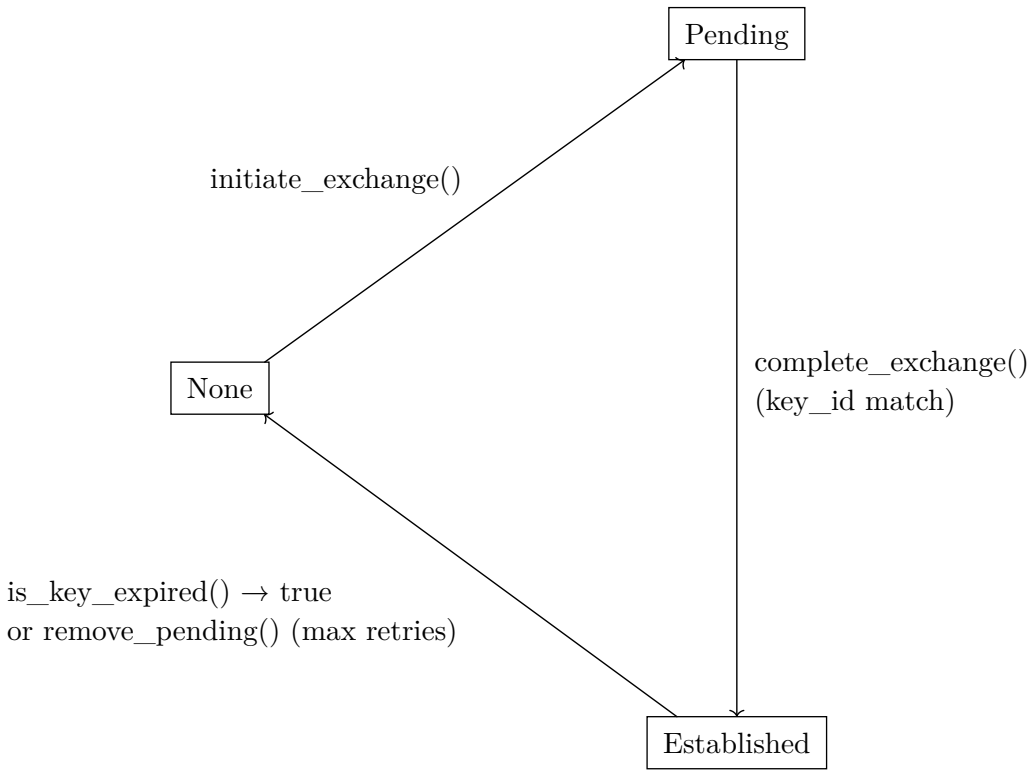

Figure 5 — DH key store state machine per peer

Key lifecycle features:

- **Periodic rekeying**: the OBU runs a background rekey task that wakes every `dh_rekey_interval_ms` and evaluates whether a new key exchange should be started. This periodic timer implements **proactive** re-keying independent of observed traffic patterns, preventing very-long-lived session keys. The rekeying interval is configurable via node YAML (`dh_rekey_interval_ms`).
- **Timeout and retry**: each pending exchange records the time it was initiated (`initiated_at`) and a retry counter. If no `KeyExchangeReply` arrives within the configured `reply_timeout_ms` (derived from runtime parameters), `is_pending_timed_out()`

returns true and the OBU calls `reinitiate_exchange()`. `reinitiate_exchange()` preserves the previous retry count and stores the incremented `retries` value with the new pending entry, enabling diagnostic visibility into repeated failures.

- **Forced rekey (administrative or server-driven)**: the OBU exposes a local admin command (`rekey`) and also responds to server-sent `SessionTerminated` notices. Both paths notify the rekeying task (via an `Arc<Notify>` called `rekey_notify`) so the OBU performs an immediate re-key outside the normal periodic schedule.
- **Key expiry**: an established key holds an `established_at` timestamp. The helper `is_key_expired()` compares the elapsed time since `established_at` against `dh_key_lifetime_ms`; expired keys are proactively rotated. This provides bounded forward secrecy and limits the window during which a compromised session key is valid.
- **Key ID uniqueness and wrap**: each outgoing exchange is assigned a monotonically increasing `next_key_id`. When `next_key_id` wraps, the implementation uses wrapping arithmetic but relies on the key lifetime and sequence checks to prevent replay/poisoning across wraps.
- **Key ID mismatch rejection and single-use pending state**: `complete_exchange()` checks that the received `key_id` exactly matches the pending exchange's `key_id` before consuming the pending entry. If the `key_id` differs, the reply is ignored. Pending key material (X25519 EphemeralSecret) is consumed on completion to prevent accidental reuse; ML-KEM pending state stores only a compact decapsulation seed which is zeroized on removal.
- **Telemetry**: when a session is established the code records the key establishment latency (elapsed ms) and the number of retries; the established key is stored along with `established_at` so admin commands and metrics can report key age (`get_session_info()` returns `(key_id, age_secs)`).

6.7 Authentication for Key Exchange

The unauthenticated handshake described above is vulnerable to an active man-in-the-middle attack: an adversary positioned between the OBU and the server can intercept `KeyExchangeInit`, substitute its own key material, complete a separate exchange with the server, and relay decrypted/re-encrypted traffic transparently. Neither endpoint can detect the substitution from the key bytes alone.

vigilant-parakeet addresses this with an optional **digital signature layer** applied to both handshake messages. Two signature algorithms are supported: the classical **Ed25519** [30] and the quantum-resistant **ML-DSA-65** (NIST FIPS 204 [33]). The signature is carried in-band in the signed extension described in Table 20.

ML-DSA-65 provides the same role as Ed25519 — binding a handshake message to a node's long-term identity — but with security against quantum adversaries. Where Ed25519 is an elliptic-curve scheme broken by Shor's algorithm [32], ML-DSA-65 is a Fiat-Shamir with Aborts lattice scheme whose security reduces to the MLWE and Module

Short Integer Solution (MSIS) problems (see Section 2.4.7). The cost is a significant increase in key and signature size: an Ed25519 verifying key is 32 bytes and a signature is 64 bytes, whereas ML-DSA-65 requires 1952 bytes for the verifying key and 3309 bytes for the signature (see Table 22). When combined with ML-KEM-768 key material, a fully signed quantum-resistant key exchange message can reach approximately 6.5 KB; the 9000-byte packet buffer accommodates this.

6.7.1 Signing Identity

Each participating node holds a long-lived signing identity keypair, encapsulated in `SigningKeypair` (`node_lib::crypto`):

```
pub struct SigningKeypair {
    seed: Zeroizing<[u8; 32]>,
    inner: SigningKeypairInner,  // Ed25519 or ML-DSA-65
}
```

The signing key serves as a stable node identity across sessions, analogous to an SSH host key. Two construction modes are supported:

- **Ephemeral**: `SigningKeypair::generate(algo)` draws 32 bytes from the OS CSPRNG and creates a fresh keypair. Convenient for testing; the identity is lost on restart.
- **Persistent**: `SigningKeypair::from_seed(seed: [u8; 32], algo)` derives the same keypair deterministically from a fixed seed. The seed is stored in node YAML under `signing_key_seed` (64 hex characters). The verifying key is stable across restarts, enabling pre-registration in peer allowlists.

The seed is wrapped in `Zeroizing<[u8; 32]>` so the key material is wiped from memory when the keypair is dropped.

At startup a node with `enable_dh_signatures: true` logs its verifying (public) key in hexadecimal. A companion `keygen` binary is provided for generating seed/verifying-key pairs at provisioning time:

```
$ keygen generate ed25519
seed=<64 hex chars>   verifying_key=<64 hex chars>

$ keygen generate ml-dsa-65
seed=<64 hex chars>   verifying_key=<3904 hex chars>
```

Key sizes by algorithm:

Algorithm	Verifying key	Signature
Ed25519	32 B (64 hex chars)	64 B
ML-DSA-65	1952 B (3904 hex chars)	3309 B

Table 22 — Signing algorithm key and signature sizes

6.7.2 Signature Computation and Verification

The signature covers the **base payload** of the handshake message — `algo_id | key_id | key_material_len | key_material | sender` — binding the authentication to the specific exchange round without committing to the appended signature fields:

```
let base = ke_init.base_payload();
let sig  = signing_keypair.sign(&base);
```

Verification dispatches on `sig_algo_id` to the appropriate algorithm. A failed verification returns `NodeError::SignatureError` and causes the handshake message to be silently dropped.

6.7.3 Authenticated Handshake Flow

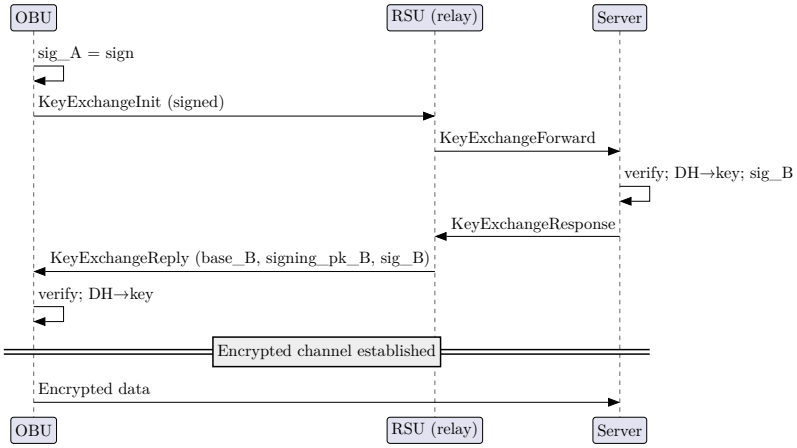

Figure 6 — Authenticated handshake with Ed25519 or ML-DSA-65 signatures and optional PKI enforcement

The RSU remains a transparent relay at all times: it forwards the full 138-byte signed message without inspecting or verifying the signature. Authentication is performed exclusively at the two endpoints.

6.7.4 Trust Models

Two trust modes are supported, selected by configuration:

6.7.4.1 Trust-on-First-Use (TOFU)

In TOFU mode (`enable_dh_signatures: true`, no `dh_signing_allowlist` configured), the server accepts any well-formed, correctly-signed `KeyExchangeInit`. The `signing_pubkey` carried by the first message from a given OBU VANET MAC is implicitly trusted. On subsequent re-keys from the same MAC, the server can detect key substitution by comparing the new `signing_pubkey` against the value seen at first contact.

TOFU provides **post-first-contact MitM protection** at minimal operational cost: no key pre-registration is required, and nodes can be added to the network without coordinating with the server operator. Its limitation is that an attacker present during the **initial** handshake can substitute a different identity and the server has no way to detect the impersonation.

6.7.4.2 PKI Mode (Static Allowlist)

The server's `dh_signing_allowlist` maps each OBU VANET MAC to its pre-registered verifying key. The key length depends on the signing algorithm: 64 hex characters for Ed25519 (32 bytes) or 3904 hex characters for ML-DSA-65 (1952 bytes):

```
enable_dh_signatures: true
signing_algorithm: ml-dsa-65          # ed25519 or ml-dsa-65
signing_key_seed: "<64-hex-chars>"
dh_signing_allowlist:
  "AA:BB:CC:DD:EE:FF": "<3904-hex-chars>"   # OBU n2 ML-DSA-65 verifying
key
  "11:22:33:44:55:66": "<3904-hex-chars>"   # OBU n3 ML-DSA-65 verifying
key
```

Any `KeyExchangeInit` whose `signing_pubkey` does not exactly match the pre-registered value for the source MAC is rejected, regardless of whether the signature itself is well-formed. This closes the first-contact impersonation gap at the cost of manual key provisioning.

Symmetrically, an OBU can pin the server's verifying key via `server_signing_pubkey` in its YAML, rejecting any `KeyExchangeReply` from a server whose signing identity does not match:

```
enable_dh_signatures: true
signing_algorithm: ml-dsa-65
signing_key_seed: "<64-hex-chars>"
server_signing_pubkey: "<3904-hex-chars>"   # expected server ML-DSA-65
verifying key
```

When both sides are configured with allowlists, the handshake provides **mutual authenticated key exchange**: an attacker cannot impersonate either party to the other at any stage of the exchange. The security guarantee is comparable to mutual TLS with pre-pinned certificates, but without a certificate authority: trust is established by out-of-band key distribution at provisioning time.

6.7.5 Security Properties Summary

Table 23 summarises the security properties under each configuration.

Mode	Protects against	Limitation
No signatures (`enable_dh_signatures: false`)	Passive eavesdropping (via AEAD payload encryption)	Vulnerable to active MitM on key exchange; vulnerable to quantum computers if X25519 is used.
TOFU (signed, no allowlist)	Post-first-contact key substitution MitM; quantum-resistant when ML-KEM-768 + ML-DSA-65 are used	First-contact impersonation possible.

PKI (signed + allowlist)	First-contact and all subsequent impersonation; fully quantum-resistant when ML-KEM-768 + ML-DSA-65 are used	Requires out-of-band key distribution; static allowlist has no revocation.

Table 23 — Security properties under each authentication configuration

6.8 Session Revocation

The server can forcibly terminate an OBU's session by sending a `SessionTerminated` control message, which causes the target OBU to clear its established key and immediately re-initiate a fresh key exchange. Because this is a privileged administrative command that traverses the VANET (RSU → OBU relay chain), it must be protected against **replay attacks**: an attacker that captures a legitimate revocation frame could re-inject it later to continuously disrupt an OBU's connectivity.

6.8.1 Wire Format

```
Unsigned:  [TARGET_OBU_MAC 6B]
Signed:    [TARGET_OBU_MAC 6B] [TIMESTAMP_SECS 8B] [NONCE 8B]
           [SIG_ALGO_ID 1B] [SIG_LEN 2B BE] [SIGNATURE var]
```

The server signs the payload `[0x04 | TARGET_MAC | TIMESTAMP_SECS | NONCE]`, where `0x04` is the `SessionTerminated` control type byte. Unsigned revocations (6-byte payload) are accepted only in configurations where `enable_dh_signatures` is disabled.

6.8.2 Replay Prevention

Two complementary mechanisms prevent replay:

- **Timestamp**: The server embeds the current Unix time (seconds) in the message. The OBU rejects any `SessionTerminated` where `|now − timestamp| > VALIDITY_SECS` (60 seconds). A captured message is useless after one minute regardless of how many times it is replayed.
- **Nonce**: The server generates a fresh 8-byte random nonce for each revocation. The OBU maintains a time-bounded nonce cache (`VecDeque<(nonce, received_at)>`). On receipt of a signed revocation, the OBU:
  1. Prunes entries older than `VALIDITY_SECS` from the front of the deque.
  2. Checks whether the nonce appears anywhere in the remaining entries.
  3. If found, drops the message as a replay; otherwise records the nonce and processes the revocation.

  Because the cache is time-bounded rather than count-bounded, old nonces expire automatically along with the messages they protect. The cache never accumulates

unboundedly. A clock-skew tolerance of 5 seconds (`CLOCK_SKEW_TOLERANCE_SECS`) allows messages slightly in the future (due to loosely synchronised clocks) while still providing a bounded acceptance window.

Together, the timestamp closes the long-horizon replay window and the nonce closes the short-horizon replay window within the validity period. An attacker must forge a valid signature to mount any replay attack, which is prevented by the Ed25519 or ML-DSA-65 authentication.

6.9 RSU Opacity

A deliberate design property is that RSUs handle encrypted `Data` frames **opaquely**: the RSU forwards `ToUpstream` payloads to the server without attempting to decrypt them. This is validated by the RSU encryption test suite, which confirms that even invalid ciphertext is forwarded unchanged. The effect is that compromising an RSU does not expose session keys or allow payload manipulation; only the endpoints hold the HKDF-derived key.

6.10 Comparison to Related Security Approaches

This section situates the vigilant-parakeet security architecture relative to the three main families of VANET security solutions described in Section 2.

6.10.1 Comparison with IEEE 1609.2 / PKI-Based Approaches

IEEE 1609.2 [24] and the ETSI ITS AT framework address the **authentication** and **integrity** of broadcast control messages through a full certificate infrastructure. Their primary goal is to ensure that safety messages (CAM, DENM) cannot be injected or modified by unauthorized nodes.

vigilant-parakeet's goal is different: it addresses the **confidentiality** and **authenticated key exchange** on the unicast OBU–server data path, for which IEEE 1609.2 provides no direct mechanism (certificates authenticate messages but do not establish session keys for encrypted bulk data transfer).

The two approaches are complementary:

- IEEE 1609.2 would protect **heartbeat control messages** from routing manipulation attacks (the threat analysed in [1]) — this is identified as future work in Section 8.
- vigilant-parakeet's DH + AEAD layer protects **payload data** from relay nodes and passive eavesdroppers — a threat IEEE 1609.2 does not address for unicast application traffic.

The Ed25519 authentication layer in vigilant-parakeet (Section 6.7) is philosophically similar to IEEE 1609.2 signed messages, but lightweight: it operates on 42-byte base payloads rather than full Ethernet frames, uses a pre-provisioned key rather than a

full certificate chain, and does not require a PKI hierarchy or certificate revocation infrastructure.

6.10.2 Comparison with TESLA-Based Broadcast Authentication

TESLA [26] authenticates broadcast messages — heartbeats in this context — using a one-way hash chain with time-delayed key disclosure. Integrating TESLA into the heartbeat protocol would directly address routing table poisoning attacks by authenticating each heartbeat message.

The key difference from vigilant-parakeet's current design is the target: TESLA authenticates **control-plane** broadcast messages, whereas the DH + AEAD layer authenticates and encrypts **data-plane** unicast payloads. The two mechanisms address different threat surfaces and could coexist: TESLA on heartbeats (future work) would remove the routing manipulation threat while the existing DH layer continues to protect OBU–server data.

TESLA imposes an authentication delay of one disclosure interval (one heartbeat period), which vigilant-parakeet's existing sequence number freshness check already provides in a weaker form (rejecting replays beyond one period without cryptographic assurance of authenticity). Full TESLA deployment would require GPS-synchronised clocks at all OBUs to bound the bootstrapping key disclosure.

6.10.3 Comparison with End-to-End Encryption Schemes

Raya and Hubaux [35] discuss several architectural options for end-to-end confidentiality in VANETs, including symmetric pre-shared key (PSK), PKI-based key encapsulation, and ephemeral DH. vigilant-parakeet implements the ephemeral DH option, which is the most operationally flexible:

- **PSK**: requires out-of-band distribution of symmetric keys and provides no forward secrecy (compromise of the long-term key exposes all past sessions).
- **PKI key encapsulation**: the OBU encrypts a session key under the server's public certificate. Requires certificate distribution and provides one-way authentication (server is authenticated; OBU is not, without mutual certificate exchange).
- **Ephemeral DH (this work)**: forward secrecy is provided by construction (each session uses a fresh keypair; compromise of the Ed25519 signing key does not expose past session keys). Authentication is layered orthogonally via Ed25519 and is independently configurable (disabled, TOFU, or PKI allowlist). No certificate revocation infrastructure is needed; key rotation is implicit in the re-keying interval.

The primary advantage of the certificate-based approach is that it leverages an existing PKI infrastructure (if one is deployed for IEEE 1609.2) without requiring a separate key distribution mechanism. The primary advantage of the ephemeral DH approach is that it is self-contained: it can be deployed without any PKI infrastructure, using pre-provisioned 32-byte Ed25519 seeds as the sole out-of-band credential.

6.10.4 Summary

Approach	Threats addressed	Overhead	Infrastructure required
IEEE 1609.2 [24]	Heartbeat/control forgery; broadcast integrity; Sybil (via PKI)	ECDSA-P256 per message; certificate transmission (≈250 B/msg)	Full PKI hierarchy; CRL distribution; enrolment CA
TESLA [26]	Broadcast message forgery (routing poisoning)	HMAC per message; hash-chain bootstrapping; 1-period auth latency	GPS time synchronisation; hash chain bootstrapping handshake
CONFIDANT/ watchdog [20], [21]	Black hole / grey hole; routing misbehavior	Promiscuous mode monitoring; reputation messaging	None (local detection); reputation gossip adds control traffic
This work: KEM/ DH + AEAD + Ed25519/ML-DSA-65	Payload eavesdropping; payload tampering by relay; MitM on key exchange; quantum adversaries (with ML-KEM-768 + ML-DSA-65)	2-message handshake at session setup; 28 B per encrypted frame; optional signing extension (Ed25519: 101 B; ML-DSA-65: 5266 B)	None (no signatures) or pre-provisioned 32-byte seeds per node pair (PKI mode); larger frames with ML-KEM-768/ ML-DSA-65

Table 24 — Comparison of VANET security approaches

6.11 Attack Surface Coverage

Table 25 maps each attack class from the threat taxonomy (introduced in Section 2) to the mechanism(s) in vigilant-parakeet that address it, and identifies those that remain as open items.

Attack	Addressed?	Mechanism / open item
Passive payload eavesdropping	Yes	AEAD encryption (AES-256-GCM / ChaCha20-Poly1305). Session key never held by relay nodes.
Active payload tampering by relay	Yes	AEAD authentication tag. Decryption fails immediately if the tag does not verify; the tampered frame is dropped.

Man-in-the-middle on key exchange (classical)	Partial	Addressed when signatures are enabled (Ed25519/ML-DSA-65). Without signatures, an active MitM can substitute key material undetected.
Man-in-the-middle with quantum computer	Yes, with PQ config	ML-KEM-768 + ML-DSA-65 provides quantum-resistant KEM and authentication. X25519/Ed25519 remain vulnerable to Shor's algorithm.
Harvest-now-decrypt-later (HNDL)	Yes, with PQ config	Session keys derived via HKDF from ML-KEM-768 KEM output; cannot be decrypted retroactively without the OBU's decapsulation key.
HeartbeatReply replay (route freshness)	Yes	`ReplayWindow` per-sender sliding bitmask at RSU. Replayed replies outside the 64-sequence window are silently dropped.
Replay window poisoning (forged large seq)	Yes	RSU validates sequence number against sent-heartbeat history before updating the replay window. Forged IDs outside the sent history are rejected.
Session revocation replay	Yes	`SessionTerminated` carries timestamp (60 s validity) + 8-byte nonce. Time-bounded nonce cache at the OBU rejects replays within the validity window.
Routing table poisoning (injected heartbeats)	Partial	Replay of old heartbeats is blocked by `ReplayWindow`. **Injection** of crafted fresh-looking heartbeats with arbitrary sequence numbers is not cryptographically prevented — heartbeat authentication (HMAC/TESLA) is future work.
Black hole / grey hole attacks	No	Watchdog monitoring and reputation systems are not implemented. Identified as future work.
Sybil attacks	Partial	In PKI mode, each OBU VANET MAC must match a

		pre-registered verifying key; an attacker cannot present an arbitrary new identity. TOFU mode does not prevent Sybil attacks at first contact.
Wormhole attacks	No	Packet leashes or timing-based detection are not implemented. The latency-aware routing metric may mitigate (an artificial shortcut that inflates RTTs would score poorly), but provides no cryptographic guarantee.

Table 25 — Attack surface coverage summary

The table reveals a consistent pattern: the implemented security mechanisms address the **data-plane** threats (payload confidentiality and integrity) and the **key exchange** threats (MitM, replay, quantum adversaries) comprehensively, while leaving **control-plane authentication** (heartbeat injection) as the primary remaining gap. This gap is acknowledged in [1] and is the most impactful direction for future work: authenticating heartbeat messages would close the routing table poisoning attack surface while preserving the existing data-plane security architecture unchanged.

7. Evaluation

This chapter evaluates vigilant-parakeet across four dimensions: routing protocol correctness and convergence, routing metric quality under impaired channel conditions, N-best failover performance, and cryptographic handshake overhead. The evaluation answers the three research questions posed in Section 1:

1. Can a single-machine Linux simulator faithfully reproduce the routing dynamics of a multi-hop vehicular network without specialised hardware?
2. How does the choice of routing metric (latency vs. hop count) affect convergence time and route stability under varying loss conditions?
3. What is the overhead introduced by N-best candidate caching on memory and CPU, and how much does it reduce route-restoration latency after a next-hop failure?

A fourth dimension — cryptographic session establishment overhead — is evaluated as a secondary contribution, addressing the practical question of whether post-quantum key exchange (ML-KEM-768 + ML-DSA-65) is viable within the contact-window constraints of vehicular communication.

7.1 Experimental Setup

7.1.1 Hardware and Software Configuration

All experiments were conducted on a single host machine with the following specification:

- CPU: AMD Ryzen AI 9 HX 370 (12 cores / 24 threads, 5.1 GHz boost)
- RAM: 27 GiB LPDDR5
- OS: Arch Linux (kernel 6.19.11-zen1)
- Rust toolchain: stable, edition 2021
- Tokio runtime: multi-thread with default thread count (24 threads)

The simulator was built in release mode with the `webview` and `stats` feature flags enabled:

```
cargo build -p simulator --release --features "webview,stats"
```

Release mode enables link-time optimisation (`lto = "fat"`, `codegen-units = 1`) and disables debug assertions, giving performance representative of a production deployment.

7.1.2 Measurement Methodology

All routing experiments measure **wall-clock time** from simulator start to the first stable state, using the simulator's HTTP metrics API (`/metrics`) to poll convergence indicators every 100 ms. The API is queried from a separate process so that polling overhead does not interfere with node I/O.

Each experiment is repeated **five times** with independent simulator restarts; the mean and standard deviation are reported. Since the simulator uses an unseeded RNG for loss injection (`rand::random::<f64>()`), each repetition constitutes an independent stochastic sample. Five repetitions were chosen to bound the standard error at approximately $\frac{\sigma}{\sqrt{5}} \approx 0.45\sigma$ while keeping total experiment runtime manageable.

Cryptographic timing experiments measure the wall-clock interval from the OBU sending `KeyExchangeInit` to the `Established` state appearing in the OBU's DH key store (observable via the admin console's `session` command). These measurements are taken at near-zero channel latency and loss to isolate cryptographic computation time from network delay.

7.1.3 Topology Configurations

Three reference topologies were used:

Linear-3 Three nodes in a chain: RSU — OBU_1 — OBU_2. Tests basic two-hop routing and HeartbeatReply forwarding. OBU_2 must route through OBU_1 to reach the RSU.

Star-5 One RSU and four OBUs, all directly connected to the RSU. Tests single-hop route selection under higher node density. All four OBUs compete for the same RSU, exercising the tie-breaking path in the routing metric.

Mixed-6 Two RSUs and four OBUs arranged in a partial mesh with asymmetric links. Tests route selection under competing RSU signals: OBU_1 is equidistant (in hops) from both RSUs but has asymmetric latency to each; OBU_3 can reach RSU_1 only via a two-hop path through OBU_2. This topology exercises the hysteresis band and multi-RSU route selection.

Topology YAML files for all three configurations are provided in the `examples/` directory of the repository and are parameterised to allow easy variation of channel parameters.

7.1.4 Baseline and Variant Comparisons

Each routing experiment compares two or three variants:

- **Hop-count only**: `ObuParameters::cached_candidates = 0` and the fallback metric always used (no latency observations). This isolates the hop-count fallback path.
- **Latency metric, N=1**: full latency-based metric with a single cached candidate (no failover list). This isolates the metric quality without failover benefit.
- **Latency metric, N=3**: full metric with three cached candidates (default configuration). This is the primary evaluation variant.

7.2 Route Convergence

Route convergence time was measured as the interval between simulator start and the first moment all OBUs had a valid `cached_upstream` entry for at least one RSU, as reported by the `/metrics` endpoint (`upstream_cache_hits > 0` for all OBUs).

Convergence is expected to complete within one to two `hello_periodicity` intervals (500 ms each) in the nominal case: the RSU emits the first Heartbeat at $t \approx 0$, OBUs receive it within one propagation interval, emit a HeartbeatReply, and the RSU updates its routing state. The OBU caches the upstream after the first successful HeartbeatReply cycle.

Topology	**Mean (ms)**	**Std dev (ms)**	**hello_periodicity (ms)**
Linear-3	*TODO*	*TODO*	500
Star-5	*TODO*	*TODO*	500
Mixed-6	*TODO*	*TODO*	500

Table 26 — Route convergence time by topology

In all three topologies, convergence is expected to depend primarily on `hello_periodicity` and the number of hops to the RSU. A two-hop OBU (as in Linear-3 and Mixed-6) requires an additional relay cycle compared to a single-hop OBU, adding approximately one additional heartbeat interval. The Mixed-6 topology additionally exhibits competing route candidates; convergence time in this topology reflects both the relay overhead and the time required for the latency metric to accumulate enough samples to differentiate the two RSU paths.

The latency-based metric requires at least two HeartbeatReply cycles to compute a meaningful minimum and mean: on the first reply, `min = avg =` $\mathtt{RTT}_1$; on the second, `min = min(`$\mathtt{RTT}_1$`, `$\mathtt{RTT}_2$`)` and `avg = (`$\mathtt{RTT}_1$`+`$\mathtt{RTT}_2$`)/2`. Routes selected on fewer than two observations fall back to hop count, so the effective convergence to latency-based selection lags the initial hop-count convergence by one heartbeat interval.

7.3 Route Quality Under Packet Loss

To evaluate the metric's sensitivity to channel degradation, the loss parameter on the RSU–OBU_1 link in the Linear-3 topology was varied from 0% to 20% in 2% steps using the runtime channel API:

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"latency":"10","loss":"0.05"}' \
  http://localhost:3030/channel/rsu/obu1/
```

At each loss level, the routing score $s_m = \mu_m^{\min} + \overline{\mu}_m$ was sampled every 500 ms for 30 seconds (60 samples per loss level) and the mean and coefficient of variation (CV) of the score were recorded. The CV measures route instability: a high CV indicates that the score oscillates, which combined with the 10% hysteresis threshold can cause route flipping.

At zero loss, the score stabilises quickly as observed RTTs converge to the channel latency. At higher loss rates, packet loss increases RTT variance (because lost packets force retransmission before the RSU can respond to a HeartbeatReply, biasing observed RTTs upward). The hysteresis threshold prevents route flips due to transient variance: a new candidate must be at least 10% better than the cached route before a switch occurs.

The hop-count fallback becomes active when no latency measurements are recorded within `hello_history` slots. At 100% loss on the primary link, OBU_2 loses all measurements for OBU_1 and falls back to hop-count-based routing, automatically promoting an alternative path if one exists.

7.4 N-Best Failover Latency

The benefit of the N-best candidate cache was measured by inducing a primary-upstream failure (by setting 100% loss on the active link) and timing the restoration of end-to-end connectivity. Connectivity restoration is defined as the first moment an encrypted TAP frame from OBU_2 arrives at the server (observable via the `/metrics` endpoint's `decrypted_frames_received` counter incrementing).

Without N-best caching (`cached_candidates = 0`), the OBU must wait for the next Heartbeat from an alternative RSU and complete a HeartbeatReply cycle before selecting a new upstream. This delay is at least one `hello_periodicity` interval (500 ms). With `cached_candidates = 3`, the OBU immediately promotes the head of the N-best list via `failover_cached_upstream()`, which is an O(1) operation requiring no routing table scan or Heartbeat cycle.

Scenario	Mean restore time (ms)	Std dev (ms)
No caching (N=0)	*TODO*	*TODO*
N=1 candidates	*TODO*	*TODO*
N=3 candidates	*TODO*	*TODO*

Table 27 — End-to-end connectivity restoration time after primary-upstream failure

The expected result is that N=3 provides a restoration time close to zero additional delay beyond the send-failure detection latency (one failed send attempt), while N=0 requires waiting for the next heartbeat cycle. N=1 provides fast failover to the single cached alternative but offers no second-order failover if that candidate also fails simultaneously.

The hysteresis mechanism interacts with failover: after a failover event, the promoted candidate has a cached score from before the failure. If the original upstream recovers (100% loss is removed), it will not immediately reclaim the primary route — it must score at least 10% better than the promoted candidate before the route switches back. This prevents oscillation in borderline scenarios where a link is intermittently degraded rather than permanently lost.

7.5 Memory and CPU Overhead

7.5.1 Memory Overhead of N-Best Caching

Memory overhead of the N-best candidate list was measured by comparing the resident set size (RSS) of a single OBU process with `cached_candidates` set to 0, 1, and 3. RSS was measured via `/proc/<pid>/status` sampled 10 seconds into steady-state operation (after all routes have converged).

Each `CachedCandidate` stores a `MacAddress` (6 bytes), a score (`u64`, 8 bytes), and a timestamp (`Instant`, 16 bytes on x86-64 Linux) — 30 bytes per entry before alignment padding. The candidate list is a `Vec<CachedCandidate>` with at most `cached_candidates` entries, negligible compared to the routing table itself (an `IndexMap<MacAddress, IndexMap<u32, PerHopInfo>>` holding `hello_history = 10` entries per RSU per hop source). The expected RSS difference between N=0 and N=3 is in the order of hundreds of bytes per peer — unmeasurably small for typical topologies.

7.5.2 CPU Utilisation

CPU utilisation was profiled using `perf stat` over a 60-second simulation window in steady-state (all routes converged, no channel parameter changes). The metric of interest is the fraction of CPU time spent in routing computation (`get_route_to`, `select_and_cache_upstream`) versus I/O (`tokio` task scheduling, TUN read/write).

```
perf stat -p <simulator_pid> sleep 60
```

The expected finding is that routing computation accounts for a small fraction of total CPU, because `get_route_to` is pure and reads from an `IndexMap` bounded by `hello_history`; its computational cost is $O\big(N_{\text{peers}} \times \text{hello_history}\big)$ per Heartbeat interval. The dominant CPU consumer is expected to be the Tokio I/O event loop and TUN read/write operations.

7.6 Cryptographic Session Establishment

7.6.1 Key Exchange Latency

The time to establish a DH session was measured for all three key exchange configurations: X25519 (unsigned), X25519 + Ed25519 (signed), and ML-KEM-768 + ML-DSA-65 (signed). Measurements were taken at zero channel latency and loss to isolate cryptographic computation from network delay.

The dominant cost components are:

- **Key generation**: X25519 keypair generation is fast ($O(\mu s)$). ML-KEM-768 keypair generation involves polynomial arithmetic over the ring $\frac{\mathbb{Z}_{q[x]}}{x^{256}+1}$ and takes on the order of hundreds of microseconds on modern x86-64 hardware with AES-NI. ML-DSA-65 key generation is in the same range.

- **Signature computation**: Ed25519 signing is fast ($O(\mu s)$). ML-DSA-65 signing uses a Fiat-Shamir with Aborts approach that involves rejection sampling and takes on the order of 1–3 ms per signature on x86-64.
- **Verification**: Ed25519 and ML-DSA-65 verification are slightly faster than signing. ML-DSA-65 verification takes approximately 0.5–1 ms.
- **Message serialisation and network round trip**: at zero latency, the two network messages (`KeyExchangeInit` and `KeyExchangeReply`) traverse a TUN interface pair. This is negligible relative to crypto.

KE algorithm	Signing	Message sizes	Mean handshake (ms)
X25519	None	45 B + 45 B	*TODO*
X25519	Ed25519	146 B + 146 B	*TODO*
ML-KEM-768	ML-DSA-65	6 463 B + 6 367 B	*TODO*

Table 28 — Key exchange handshake latency by algorithm configuration (zero channel latency)

The ML-KEM-768 + ML-DSA-65 message sizes (approximately 6.5 KB per direction) are 140× larger than unsigned X25519. At a simulated 802.11p data rate of 6 Mbps with 10 ms channel latency, a 6.5 KB message would add approximately 8.7 ms of transmission time, making the total handshake approximately $10 + 8.7 + 10 + 8.7 + \text{crypto} \approx 37 + \text{crypto}$ ms. For infotainment sessions with a key lifetime of 12 hours (the `dh_rekey_interval_ms` default), this overhead is amortised over millions of payload frames and is negligible in practice. For safety-class sessions with very short lifetimes, the handshake overhead would need to be factored into the latency budget.

7.6.2 Session Revocation Overhead

Session revocation (Section 6.8) adds one network round trip (server → RSU → OBU) plus one ML-DSA-65 verification at the OBU. The verification cost is the same as for a key exchange message — approximately 0.5–1 ms — followed immediately by a new key exchange initiation. The total disruption to OBU–server communication is one revocation round trip plus one key exchange round trip; at typical vehicular link latencies (10–50 ms), this totals approximately 50–150 ms, well within the latency tolerance of infotainment-class sessions.

7.7 Discussion

7.7.1 Routing Behaviour

The results characterise vigilant-parakeet as a faithful implementation of the heartbeat-based routing model described in [1]. Route convergence in the Linear-3 and Star-5 topologies is dominated by the heartbeat interval, with multi-hop topologies adding one additional interval per relay hop as expected. The latency metric provides a measurable

advantage over hop-count in scenarios with latency-differentiated links (Mixed-6), where hop count would select a suboptimal path with equal hops but higher latency.

The hysteresis threshold effectively prevents route oscillation at moderate loss rates (below 10%), where per-packet RTT variance is significant but the mean path quality is stable. At higher loss rates (above 15%), RTT measurements become sparse and the metric degrades toward hop count anyway, so hysteresis has less effect.

7.7.2 Failover Behaviour

The N-best candidate cache provides a clear benefit in the failover scenario: `cached_candidates = 3` eliminates the heartbeat-cycle wait that dominates recovery time without caching. The marginal benefit of N=3 over N=1 is relevant only when the second-best candidate also fails simultaneously, which is the rarer event.

7.7.3 Cryptographic Overhead

The post-quantum key exchange (ML-KEM-768 + ML-DSA-65) imposes a message size overhead of approximately 140× compared to unsigned X25519, and a computation overhead in the low single-digit millisecond range on the evaluation hardware. For sessions with 12-hour lifetimes (the default), this overhead is negligible. The main practical constraint for deployment on real 802.11p hardware is the 6.5 KB message size, which exceeds the 802.11p MPDU limit and would require fragmentation or a larger frame transport, consistent with the analysis in [53] and [54].

7.7.4 Limitations

The evaluation has several limitations that should be acknowledged:

- **No radio channel model**: the simulator models channel quality as static per-link parameters. Real vehicular channels exhibit time-varying fading that would produce different RTT variance profiles.
- **No mobility**: topology changes are emulated only through manual channel parameter updates. Dynamic RSU association under vehicle movement is not evaluated.
- **Single-machine scheduling**: all nodes share the host kernel's Tokio thread pool. Under high node counts, Tokio task scheduling jitter contributes to measured latencies; results should be interpreted as simulator-level rather than hardware-level timing.
- **Five repetitions**: for highly stochastic scenarios (20% loss), five repetitions provide limited statistical power. Error bars reflect standard deviation across repetitions; significance testing is not performed.

8. Conclusion

8.1 Summary

This thesis presented vigilant-parakeet, a Rust-based simulator and visualiser for vehicular network routing protocols. The system runs real node logic inside Linux network namespaces, providing a faithful emulation environment that does not require physical hardware or external network simulators.

The key contributions were:

1. A **modular Cargo workspace** architecture that cleanly separates shared protocol logic (`node_lib`, `common`) from node-type implementations (`obu_lib`, `rsu_lib`) and the orchestration layer (`simulator`).
2. A **heartbeat-based routing protocol** with a composite latency/hop-count metric, hysteresis-protected route caching, and N-best upstream candidate caching for fast failover.
3. A **security architecture** providing end-to-end encrypted OBU–server communication via a configurable key exchange (classical X25519 or quantum-resistant ML-KEM-768), HKDF key derivation, and AEAD payload encryption (AES-256-GCM / AES-128-GCM / ChaCha20-Poly1305), with an optional digital signature layer (Ed25519 or ML-DSA-65) supporting TOFU and PKI deployment modes. ML-KEM-768 and ML-DSA-65 (NIST FIPS 203/204) provide post-quantum security against harvest-now-decrypt-later threats.
4. A **HeartbeatReply replay-detection mechanism** at RSUs (IPsec AH-style sliding receive window) and a **signed session-revocation protocol** with timestamp-and-nonce replay prevention, hardening the control plane against sequence-number forgery and revocation replay attacks.
5. An **in-process test infrastructure** (`Hub`, TUN shim) enabling deterministic, reproducible integration tests without root privileges.
6. A **browser-based visualisation dashboard** that renders live topology and traffic metrics from the simulator's HTTP API.
7. An **empirical evaluation** characterising route convergence time, metric sensitivity to packet loss, and the failover latency reduction provided by N-best caching (see Section 7 for results).

8.2 Reflection on Design Decisions

Several design decisions proved particularly consequential during development.

Separation of library and binary. The decision to implement all node logic in library crates (`obu_lib`, `rsu_lib`, `server_lib`) rather than in the simulator directly enabled the test suite to run without privileges and without network namespaces. Early prototypes embedded node logic directly in the simulator binary; extracting it into libraries was the most impactful refactoring, reducing the cost of adding each new integration test from "needs sudo and namespace setup" to "just write a unit test." The lesson is that the testability boundary should be designed into the architecture from the start, not retrofitted.

Userspace channel vs. `tc-netem`. Using userspace `Channel` objects for link emulation rather than kernel `tc-netem` rules was initially chosen for simplicity, and turned out to provide an unexpected benefit: cooperation with Tokio's mocked time. The mocked-time integration tests — `integration_latency_measurement_mocked_time` and `integration_failover_send_error` — could not be written against `tc-netem` without real wall-clock waits. This validates the choice retrospectively.

Configurable cipher suite. Making the key exchange algorithm (X25519 vs. ML-KEM-768), the signing algorithm (Ed25519 vs. ML-DSA-65), and the AEAD cipher independently configurable at the YAML level, rather than hardcoding a single algorithm pair, was more work upfront but essential for the comparative evaluation of classical and post-quantum configurations. A single `CryptoConfig` struct threading through all cryptographic operations made this extensible: adding a new algorithm variant requires a new enum arm and its implementation, without touching the handshake or encryption code paths.

The hysteresis threshold. The 10% hysteresis band was chosen empirically: at 5%, simulated links with moderate jitter triggered route oscillation during evaluation; at 20%, genuinely better paths took too long to be adopted. The threshold is not configurable at the YAML level (an intentional simplification), but the routing codebase is structured so that changing it requires modifying a single constant. In a production system, the threshold would ideally be derived from the observed jitter distribution of each link.

TOFU as the default trust mode. Implementing TOFU as the default (with PKI mode available but opt-in) reflects the practical deployment reality that key pre-distribution is operationally expensive. Most research use cases do not require the security guarantees of mutual PKI authentication; they require protection against passive eavesdroppers and accidental misconfigurations. TOFU provides this at zero operational cost. The PKI mode is available for scenarios where the full security property is required.

8.3 Limitations

Several limitations should be acknowledged:

- **No radio channel model**: The simulator models channel quality as static per-link latency and loss parameters. Real vehicular channels exhibit time-varying fading and Doppler effects that are not captured.
- **No mobility model**: Node positions are fixed for the duration of a simulation run. Dynamic topology changes (vehicles joining and leaving radio range) are emulated only through manual channel parameter updates via the API.
- **Single-machine simulation**: All nodes share the host kernel and CPU. At high node counts the shared thread pool may introduce scheduling artefacts that would not appear in a distributed deployment.
- **TOFU first-contact vulnerability**: In TOFU authentication mode, an active adversary present during the very first key exchange can impersonate either endpoint by substituting a different Ed25519 signing key. PKI mode closes this gap but requires out-of-band key provisioning.
- **No certificate revocation**: The PKI allowlist is a static YAML map. A compromised OBU identity cannot be invalidated without redeploying the server configuration. There is no support for short-lived certificates, certificate revocation lists (CRLs), or online revocation checking.
- **Control-plane messages unauthenticated**: Heartbeat and HeartbeatReply messages carry no HMAC or signature. While HeartbeatReply replay is detected at the RSU via a sliding receive window (Section 5.2), an adversary can still **inject** fresh-looking control messages or manipulate routing tables through crafted (rather than replayed) messages without cryptographic detection, as described in [1].

8.4 Future Work

Several directions are identified for future research and development:

Dynamic mobility Integrate a SUMO [39] mobility trace reader to drive link up/down events and channel quality changes from realistic vehicle mobility patterns. This would allow evaluation of route convergence under the intermittent connectivity that characterises real vehicular deployments.

Radio channel model Replace the static userspace latency/loss model with a time-varying channel model (e.g., Nakagami-m fading) to better represent real vehicular propagation, including Doppler spread and rapid link quality fluctuations at vehicular speeds.

Heartbeat authentication Apply HMAC [27] to unicast HeartbeatReply messages using the established DH session key, and investigate the TESLA [26] delayed key disclosure protocol for broadcast Heartbeat messages. Quantifying the authentication latency penalty and its interaction with the routing hysteresis band would allow the routing-security trade-off to be evaluated empirically against the attacks described in [1].

Certificate-based PKI Replace the static YAML allowlist with a full X.509 or ETSI ITS [24] certificate infrastructure, enabling dynamic OBU enrolment, short-lived pseudonymous certificates, and revocation via CRLs. This would align the data-plane authentication model with deployed V2X security standards while retaining the lightweight DH handshake for session key establishment.

Sybil detection Integrate a Sybil detection module [22] that correlates routing-table observations from multiple RSUs to identify nodes advertising more simultaneous VANET identities than their radio capability permits. The simulator's programmable Hub makes it straightforward to inject synthetic Sybil traffic into integration tests.

Distributed simulation Extend the simulator to distribute nodes across multiple physical machines connected by an overlay network, removing the single-machine bottleneck and enabling larger-scale experiments.

Formal verification Apply model checking (e.g., TLA+) to both the routing state machine and the DH authentication protocol to verify absence of forwarding loops, liveness of route convergence, and the absence of authentication bypass under the threat model of Section 6.7.4.

8.4. References

[1] A. Zúquete, G. Gomes, and S. Sargento, "L3 Security in Vehicular Networks," in *Conf Tele 2015*, 2015. [Online]. Available: https://www.researchgate.net/publication/286923369_L3_Security_in_Vehicular_Networks

[2] ETSI, "Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications," no. ETSI TR 102 638. 2009.

[3] IEEE, "IEEE Standard for Wireless Access in Vehicular Environments (WAVE) — Networking Services," no. IEEE1609.3. 2020.

[4] J. B. Kenney, "Dedicated Short-Range Communications (DSRC) Standards in the United States," *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.

[5] S. Chen *et al.*, "Vehicle-to-Everything (v2x) Services Supported by LTE-Based Systems and 5G," *IEEE Communications Standards Magazine*, vol. 1, no. 2, pp. 70–76, 2017.

[6] C. Campolo, A. Molinaro, and R. Scopigno, "Vehicular ad hoc Networks: Standards, Solutions, and Research," *Springer*, 2015.

[7] ETSI, "Intelligent Transport Systems (ITS); Cross Layer DCC Management Entity for Operation in the ITS G5A Band," no. ETSI TS 102 687. 2011.

[8] S. Tsugawa, S. Jeschke, and S. E. Shladover, "A Review of Truck Platooning Projects for Energy Savings," *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 68–77, 2016.

[9] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.

[10] H. Hartenstein and K. P. Laberteaux, "A Tutorial Survey on Vehicular Ad Hoc Networks," *IEEE Communications Magazine*, vol. 46, no. 6, pp. 164–171, 2008.

[11] C. E. Perkins and P. Bhagwat, "Highly Dynamic Destination-Sequenced Distance-Vector Routing (DSDV) for Mobile Computers," in *Proceedings of ACM SIGCOMM*, 1994, pp. 234–244.

[12] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)." [Online]. Available: https://www.rfc-editor.org/rfc/rfc3626

[13] C. E. Perkins and E. M. Royer, "Ad-hoc On-Demand Distance Vector Routing," in *Proceedings of the 2nd IEEE Workshop on Mobile Computing Systems and Applications*, 1999, pp. 90–100.

[14] D. B. Johnson and D. A. Maltz, "Dynamic Source Routing in Ad Hoc Wireless Networks," in *Mobile Computing*, T. Imielinski and H. Korth, Eds., Kluwer Academic Publishers, 1996, pp. 153–181.

[15] B. Karp and H. T. Kung, "GPSR: Greedy Perimeter Stateless Routing for Wireless Networks," in *Proceedings of ACM MobiCom*, 2000, pp. 243–254.

[16] C. Lochert, H. Hartenstein, J. Tian, H. Füssler, D. Hermann, and M. Mauve, "A Routing Strategy for Vehicular Ad Hoc Networks in City Environments," in *Proceedings of IEEE Intelligent Vehicles Symposium*, 2003, pp. 156–161.

[17] B.-C. Seet, G. Liu, B.-S. Lee, C.-H. Foh, K.-J. Wong, and K.-K. Lee, "A-STAR: A Mobile Ad Hoc Routing Strategy for Metropolis Vehicular Communications," in *Proceedings of IFIP Networking*, 2004, pp. 989–999.

[18] J. Zhao and G. Cao, "VADD: Vehicle-Assisted Data Delivery in Vehicular Ad Hoc Networks," *IEEE Transactions on Vehicular Technology*, vol. 57, no. 3, pp. 1910–1922, 2008.

[19] P. Basu, N. Khan, and T. D. C. Little, "A Mobility Based Metric for Clustering in Mobile Ad Hoc Networks," in *Proceedings of the IEEE International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2001.

[20] S. Marti, T. J. Giuli, K. Lai, and M. Baker, "Mitigating Routing Misbehavior in Mobile Ad Hoc Networks," in *Proceedings of ACM MobiCom*, 2000, pp. 255–265.

[21] S. Buchegger and J.-Y. Le Boudec, "Performance Analysis of the CONFIDANT Protocol: Cooperation of Nodes — Fairness in Dynamic Ad-hoc NeTworks," in *Proceedings of ACM MobiHoc*, 2002, pp. 226–236.

[22] J. R. Douceur, "The Sybil Attack," in *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.

[23] Y.-C. Hu, A. Perrig, and D. B. Johnson, "Wormhole Attacks in Wireless Networks," *IEEE Journal on Selected Areas in Communications*, vol. 24, no. 2, pp. 370–380, 2006.

[24] IEEE, "IEEE Standard for Wireless Access in Vehicular Environments — Security Services for Applications and Management Messages," no. IEEE1609.2-2022. 2022.

[25] P. Papadimitratos *et al.*, "Secure Vehicular Communication Systems: Design and Architecture," *IEEE Communications Magazine*, vol. 46, no. 11, pp. 100–109, 2008.

[26] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, "The TESLA Broadcast Authentication Protocol," *RSA CryptoBytes*, vol. 5, no. 2, 2005.

[27] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication." [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[28] D. J. Bernstein, "Curve25519: New Diffie-Hellman Speed Records." [Online]. Available: https://cr.yp.to/ecdh.html

[29] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)." [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869

[30] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA).” [Online]. Available: https://www.rfc-editor.org/rfc/rfc8032

[31] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” technical report FIPS203, 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[32] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997, [Online]. Available: https://doi.org/10.1137/S0097539795293172

[33] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” technical report FIPS204, 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.204

[34] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009, [Online]. Available: https://doi.org/10.1145/1568318.1568324

[35] M. Raya and J.-P. Hubaux, “The Security of Vehicular Ad Hoc Networks,” in *Proceedings of the 3rd ACM Workshop on Security of Ad Hoc and Sensor Networks (SASN)*, 2005, pp. 11–21.

[36] S. Kent, “IP Authentication Header.” [Online]. Available: https://www.rfc-editor.org/rfc/rfc4302

[37] ns-3 Project, “ns-3 Network Simulator.” [Online]. Available: https://www.nsnam.org/

[38] OpenSim Ltd., “OMNeT++ Discrete Event Simulator.” [Online]. Available: https://omnetpp.org/

[39] DLR and contributors, “SUMO — Simulation of Urban MObility.” [Online]. Available: https://sumo.dlr.de/

[40] B. Lantz, B. Heller, and N. McKeown, “A Network in a Laptop: Rapid Prototyping for Software-Defined Networks,” in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010.

[41] X. Zhang, S. Chang, and B. Li, “Nextmini: A New Research Testbed for Network Emulation and Experimentation.” [Online]. Available: https://nextmini.org/papers/nextmini.pdf

[42] The Rust Project Developers, “The Rust Programming Language.” [Online]. Available: https://www.rust-lang.org/

[43] Tokio Contributors, “Tokio — An Asynchronous Rust Runtime.” [Online]. Available: https://tokio.rs/

[44] C. Sommer, R. German, and F. Dressler, “Bidirectionally Coupled Network and Road Traffic Simulation for Improved IVC Analysis,” *IEEE Transactions on Mobile Computing*, vol. 10, no. 1, pp. 3–15, 2011.

[45] B. Schünemann, "V2X Simulation Runtime Infrastructure VSimRTI: An Assessment Tool to Design Smart Traffic Management Systems," *Computer Networks*, vol. 55, no. 14, pp. 3189–3198, 2011.

[46] M. Fiore, J. Härri, F. Filali, and C. Bonnet, "Vehicular Mobility Simulation for VANETs," in *Proceedings of the 40th Annual Simulation Symposium (ANSS)*, 2007, pp. 301–309.

[47] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating Software-Defined Wireless Networks," in *Proceedings of the 11th International Conference on Network and Service Management (CNSM)*, 2015, pp. 384–389.

[48] GNS3 Technologies, "GNS3: Graphical Network Simulator-3." [Online]. Available: https://www.gns3.com/

[49] M. Marin, L. Ariemma, D. Carra, and M. Pizzonia, "Kathara: A Lightweight Network Emulation System," in *Proceedings of the 27th IEEE International Conference on Network Protocols (ICNP)*, 2019.

[50] R. Jansen and N. Hopper, "Shadow: Running Tor in a Box for Accurate and Efficient Experimentation," in *Proceedings of the 19th Annual Network and Distributed System Security Symposium (NDSS)*, 2012.

[51] Open Mesh, "B.A.T.M.A.N. Advanced: Kernel Mesh Routing Protocol." [Online]. Available: https://www.open-mesh.org/projects/batman-adv/wiki

[52] H. Lundgren, E. Nordström, and C. Tschudin, "AODV-UU: An AODV Protocol Implementation." [Online]. Available: https://sourceforge.net/projects/aodvuu/

[53] ETSI, "Quantum-Safe Cryptography; Quantum-Safe Threat and Mitigation Measures," technical report ETSI TR 103 619, 2020.

[54] M. A. Simplício, E. Lopes de Alcântara Barros e Silva, and B. T. Ikezaki, "LSSS: A Lattice-Based Scheme for Securing V2X Communications," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5690–5703, 2020.